TrpB MetaDynamics Replication Tutorial

Project: Reproducing the COMM domain conformational landscape of *Pyrococcus furiosus* TrpB
Reference: Maria-Solano, Iglesias-Fernandez & Osuna, *JACS* 2019, 141, 13049–13056
DOI: 10.1021/jacs.9b03646
Supporting Information: ja9b03646_si_001.pdf
Author: Zhenpeng Liu (@ad.unc.edu), UNC Chapel Hill
Last updated: 2026-04-02

> **IMPORTANT**: Throughout this tutorial, paths use `$WORK` as shorthand for your working directory on Longleaf. Before starting, set this variable:
>
> ```
> export WORK=/work/users/<first-letter>/<first-two-letters>/<your-onyen>/AnimaLab
> # Example: export WORK=/work/users/j/jo/johndoe/AnimaLab
> ```
>
> Replace `<your-onyen>` with your actual UNC ONYEN. Directory names are arbitrary – you can use any project name instead of "AnimaLab".

How to Read This Document

- Every command is **copy-pasteable** with full absolute paths.
- Every input file is written inline with `cat > filename << 'EOF' ... EOF`.
- Every parameter carries a **source tag**: `[SI p.S2]`, `[SI p.S3]`, `[Caulkins 2014]`, or `[operational default]`.
- Phases 0–5 are **verified** on UNC Longleaf as of 2026-04-01.
- Phases 6–9 are marked **[DRAFT – not yet verified]**.
- Every step ends with a **verification command** (`grep`, `ls -lh`, `wc -l`).

Table of Contents

1. Prerequisites
2. Phase 0: Environment Setup (~30 min)
3. Phase 1: Download & Prepare PDB Structures (~10 min)
4. Phase 2: PLP Cofactor Parameterization (~2–3 days)
5. Phase 3: Build System with tleap (~5 min)
6. Phase 4: Classical MD – Prep Pipeline (~24 hrs)
7. Phase 5: Classical MD – Production 500 ns (~3 days)
8. Phase 6: AMBER to GROMACS Conversion (DRAFT) (~30 min)
9. Phase 7: Path CV Construction (DRAFT) (~10 min)
10. Phase 8: Well-Tempered MetaDynamics (~2–3 days)
11. Phase 9: FES Reconstruction (DRAFT) (~1 hr)
12. Appendix A: Parameter Reference
13. Appendix B: Troubleshooting

Prerequisites

What You Need Before Starting

This tutorial assumes **nothing is set up yet**. You need:

1. **A computer with a terminal** (Mac/Linux terminal, or Windows with WSL or PuTTY)
2. **An HPC cluster account** with GPU access (see below for how to get one)
3. **Internet access** for downloading PDB files and software

How to Get HPC Access

This tutorial uses **UNC Longleaf** as an example. If you are at a different institution, adapt the module names and paths accordingly.

UNC Longleaf (UNC students/researchers):

1. Go to https://help.rc.unc.edu/getting-started/
2. Click "Request an Account" — you need a valid UNC ONYEN
3. Fill out the form: select "Longleaf" as the cluster, describe your research briefly
4. Wait for approval email (typically 1–2 business days)
5. Once approved, connect via SSH: `ssh <your-onyen>@longleaf.unc.edu`

Other institutions:

- Most universities have an HPC cluster. Search "[your university] HPC" or "[your university] research computing"
- You typically need: a faculty sponsor, a brief project description, and a university ID
- Common job schedulers: SLURM (most common, used here), PBS/Torque, SGE
- GPU access may require a separate request (e.g., `gpu` partition access on Longleaf)

How to Connect via SSH

> **Shell basics for beginners**: In this tutorial, lines starting with # are comments – the computer ignores them, they're just notes for you. Lines starting with $ or without any prefix are commands you type. The $ symbol represents your terminal prompt; don't type it, just type what comes after it.
>
> **> vs >>**: > creates a new file (or overwrites an existing one). >> appends to an existing file without deleting its contents. Think of > as "start fresh" and >> as "add to the end".

```
# From your local terminal (Mac/Linux):
ssh <your-onyen>@longleaf.unc.edu
# Enter your password when prompted

# For persistent connections (recommended, avoids re-entering password):
# Add to ~/.ssh/config on your LOCAL machine:
# (See "heredoc" note below)
cat >> ~/.ssh/config << 'EOF'
Host longleaf
  HostName longleaf.unc.edu
  User <your-onyen>
```

```
  ControlMaster auto
  ControlPath /tmp/ssh-%r@%h:%p
  ControlPersist 4h
EOF
# Then you can simply type: ssh longleaf
```

> **What is `cat > file << 'EOF'`?** This is called a "heredoc" – it creates a file by typing its contents directly in the terminal. Everything between `<< 'EOF'` and the closing `EOF` on its own line gets written into the file. It's the same as opening a text editor, typing the content, and saving. We use it so you can copy-paste the entire block at once. (The >> variant appends to an existing file instead of overwriting it.)

Transferring files between your computer and the cluster:

```
# Upload from local to cluster:
scp local_file.txt <your-onyen>@longleaf.unc.edu:$WORK/

# Download from cluster to local:
scp <your-onyen>@longleaf.unc.edu:$WORK/file.txt ./
```

Software Overview

None of these need to be pre-installed. Phase 0 walks through installing each one.

Software	Version	What It Does	How to Get It
AMBER	24p3	Classical molecular dynamics engine `[SI p.S2]`	`module load` on HPC (pre-installed by admin)
AmberTools	24p3	Force field tools: antechamber, tleap, parmchk2 `[SI p.S2]`	Bundled with AMBER; or free download: https://ambermd.org/AmberTools.php
Gaussian	16c02	Quantum chemistry for RESP charge calculation `[SI p.S2]`	`module load` on HPC (licensed; ask admin)
GROMACS	2026.0	MetaDynamics MD engine `[SI p.S3]`	Install via conda in Phase 0 (https://www.gromacs.org/)
PLUMED	2.9	Enhanced sampling plugin `[SI p.S3]`	Install via conda in Phase 0 (https://www.plumed.org/)
Anaconda/conda	2024.02	Package manager for GROMACS+PLUMED	`module load` on HPC; or download: https://docs.conda.io/en/latest/miniconda.ht
Python	3.10	Analysis scripts	Comes with conda environment
reduce	(bundled)	Add hydrogens to PDB structures	Bundled with AmberTools

PyMOL	any	(Optional) Structure visualization	https://pymol.org/ or `conda install pymol-open-source`

Key Reference Documents

- **SI pp. S2–S3**: System preparation, force field, MD protocol
- **SI pp. S3–S4**: MetaDynamics parameters, path CV construction
- **Caulkins et al.**[1]: PLP protonation states (net charge = -2)
- **Wang et al.**[2]: GAFF force field
- **Bayly et al.**[3]: RESP charge fitting

Disk Space Requirements

Data	Approximate Size
PDB files + force field parameters	~100 MB
Classical MD trajectory (500 ns)	~11 GB
MetaDynamics (10 walkers x 50 ns)	~5 GB
Analysis output	~500 MB
Total	**~17 GB**

Make sure your HPC work directory has enough quota. On Longleaf, `/work/` provides 10 TB.

Phase 0: Environment Setup (~30 min)

> **Goal**: Install and verify all required software on the HPC cluster. **Where**: All commands in this section run on the HPC (SSH in first).

Step 0.0: Connect to the cluster

```
ssh <your-onyen>@longleaf.unc.edu
# You are now on the login node.
```

Step 0.1: Create project directory structure

On Longleaf, your home directory has limited space (~50 GB). Use `/work/` for large files.

> **What is \ at the end of a line?** The backslash tells the shell "this command continues on the next line." It's used to break long commands across multiple lines for readability. When you copy-paste, the shell treats all the \-connected lines as one single command.

[1] Caulkins, B. G.; Bastin, B.; Yang, C.; Neubauer, T. J.; Young, R. P.; Hilario, E.; Huang, Y.-m. M.; Chang, C.-e. A.; Fan, L.; Dunn, M. F.; Marsella, M. J.; Mueller, L. J. *J. Am. Chem. Soc.* **2014**, *136*, 12824-12827.

[2] Wang, J.; Wolf, R. M.; Caldwell, J. W.; Kollman, P. A.; Case, D. A. *J. Comput. Chem.* **2004**, *25*, 1157-1174.

[3] Bayly, C. I.; Cieplak, P.; Cornell, W. D.; Kollman, P. A. *J. Phys. Chem.* **1993**, *97*, 10269-10280.

```
# Set your working directory. The path structure depends on your HPC.
# On Longleaf: /work/users/<first-letter>/<first-two-letters>/<onyen>/
# The project folder name ("AnimaLab" below)
# is arbitrary -- call it whatever you want.
export WORK=/work/users/\
  <first-letter>/<first-two-letters>/\
  <your-onyen>/AnimaLab

# Create the full directory tree
# These subdirectory names are conventions,
# not requirements -- rename as you see fit.
mkdir -p $WORK/{\
  structures,\
  parameters/{plp_structures,resp_charges,mol2,frcmod},\
  systems/pftrps_ain,\
  scripts/{amber_md,gromacs_metad},\
  runs/pftrps_ain_md,\
  logs,\
  analysis}
```

Verification:

```
ls -d $WORK/*/
# Should list: analysis, logs, parameters, runs, scripts, structures, systems
```

Step 0.2: Check what software is already available

Before installing anything, see what your HPC already provides:

> **What is `module load`?** HPC clusters have many software packages installed, but they're not all active by default (to avoid conflicts). `module load amber/24p3` activates AMBER version 24p3, adding its programs to your PATH so you can use them. `module avail` lists all available software.

```
module avail amber       # Look for amber/24p3 or similar
module avail gaussian    # Look for gaussian/16 or similar
module avail anaconda    # Look for anaconda or miniconda
module avail gromacs     # Check if GROMACS exists (may lack PLUMED support)
```

> **If AMBER is not available**: Contact your HPC admin to request installation. AmberTools (which includes antechamber, tleap, parmchk2) is free: https://ambermd.org/AmberTools.php. Full AMBER (with pmemd.cuda) requires a license (~$500 academic): https://ambermd.org/GetAmber.php
>
> **If Gaussian is not available**: This is licensed commercial software. Options: 1. Ask your HPC admin (most universities have a site license) 2. Use a collaborator's Gaussian access 3. Use ORCA as a free alternative (https://www.faccts.de/orca/) — input format differs 4. Use Psi4 (open-source, https://psicode.org/) — also requires adapting input files
>
> **If Anaconda/conda is not available**: Download Miniconda directly:

```
wget https://repo.anaconda.com/miniconda/Miniconda3-latest-Linux-x86_64.sh
bash Miniconda3-latest-Linux-x86_64.sh -b -p $HOME/miniconda3
eval "$($HOME/miniconda3/bin/conda shell.bash hook)"
conda init bash
```

Step 0.3: Install PLUMED and GROMACS via conda

The HPC's system GROMACS (if any) typically does **not** include PLUMED support. We install our own via conda-forge, which provides pre-compiled GROMACS with the PLUMED patch already applied.

What is conda-forge? A community-maintained collection of pre-built scientific software. Website: https://conda-forge.org/

What is PLUMED? A plugin for MD engines that adds enhanced sampling methods (metadynamics, umbrella sampling, etc.). Website: https://www.plumed.org/ — Documentation: https://www.plumed.org/doc

What is GROMACS? A high-performance molecular dynamics engine for biomolecular simulations. Website: https://www.gromacs.org/ — Manual: https://manual.gromacs.org/

```
# Load the conda module (adjust module name for your HPC)
module load anaconda/2024.02

# Create a dedicated environment
# (keeps dependencies isolated from other projects)
# "trpb-md" is just a name -- call it whatever you want
conda create -n trpb-md python=3.10 -y
```

> **If this fails:** If `conda` command not found, see Step 0.2 for installing Miniconda. If you get a permissions error, make sure you are creating the environment in a writable location (e.g., `--prefix /work/.../conda/envs/trpb-md`).
>
> **What is `$()`?** This runs a command inside the parentheses and substitutes its output. For example, `echo "Today is $(date)"` runs `date`, gets the current date, and inserts it into the echo command. The older syntax uses backticks `` `command` `` but `$()` is preferred because it's easier to read.

```
# Activate the environment (must use eval for non-interactive shells)
eval "$(conda shell.bash hook)"
conda activate trpb-md

# Install PLUMED + GROMACS from conda-forge
# This gives you GROMACS pre-patched with PLUMED support
conda install -c conda-forge plumed py-plumed gromacs -y
```

> **If this fails:** If installation hangs or fails with dependency conflicts, try: `conda install -c conda-forge plumed py-plumed gromacs --solver=libmamba`. If that still fails, try creating a fresh environment with: `mamba create -n trpb-md -c conda-forge python=3.10 plumed py-plumed gromacs -y`.

```
# Install analysis tools
conda install -c conda-forge mdanalysis biopython -y
```

Verification (all must succeed):

> **What is | (pipe)?** The vertical bar | sends the output of one command as input to the next command. Think of it as a conveyor belt: `command1 | command2` means "run command1, then feed its output into command2". For example, `cat file.txt | grep "hello"` reads the file and then searches for "hello" in it.
>
> **What is 2>&1?** Programs produce two streams of output: standard output (stdout, 1) and error messages (stderr, 2). 2>&1 merges them into one stream so both normal output and errors go to the same place (e.g., a log file). Combined with `| tee logfile`, it saves everything to a file while also showing it on screen.

```
plumed --version                                # Expected: 2.9.x
gmx --version 2>&1 | grep "GROMACS version"  # Expected: GROMACS version 2026.0
python -c "import MDAnalysis; print('MDAnalysis OK')"
echo "PLUMED + GROMACS installation OK"
```

Step 0.4: Verify AMBER and AmberTools

```
module load amber/24p3    # Adjust for your HPC
```

> **If this fails:** If `module load amber/24p3` gives "module not found", run `module avail amber` to see available versions. Your HPC may use a different module name (e.g., `amber/22`, `ambertools/24`). Contact your HPC admin if no AMBER module exists.

```
# Check that all required tools are available:
which pmemd.cuda     # GPU-accelerated MD engine
which antechamber    # Atom type assignment for non-standard residues
which tleap          # System builder (solvation, counterions, topology)
which parmchk2       # Generate missing force field parameters
which reduce         # Add hydrogens to PDB structures
which cpptraj        # Trajectory analysis
```

If any `which` command returns "not found", the AMBER installation is incomplete — contact your HPC admin.

Step 0.5: Verify Gaussian availability

```
module load gaussian/16c02    # Adjust module name for your HPC
which g16
# Expected: a valid path (e.g., /nas/longleaf/apps/gaussian/16c02/g16)
```

> **Note**: If `module avail gaussian` shows nothing, see the alternatives listed in Prerequisites above.

Step 0.6: Save environment for future login sessions

```
# Add to ~/.bashrc so modules load automatically when you log in
cat >> ~/.bashrc << 'BASHEOF'

# === TrpB MetaDynamics project environment ===
module load anaconda/2024.02
module load amber/24p3
BASHEOF

source ~/.bashrc
```

Final verification — log out and log back in:

> **What are head and tail?** `head -N` shows the first N lines of a file. `tail -N` shows the last N lines. Useful for peeking at large files without opening them entirely. `tail -1` shows just the last line.
>
> **What is &&?** It means "run the next command only if the previous one succeeded." `command1 && command2` runs command2 only if command1 finished without errors. If command1 fails, command2 is skipped.

```
exit
ssh <your-onyen>@longleaf.unc.edu
conda activate trpb-md
plumed --version && \
  gmx --version 2>&1 | head -2 && \
  which pmemd.cuda && \
  echo "All software OK"
```

Phase 1: Download & Prepare PDB Structures (~10 min)

> **Goal**: Obtain crystal structures for the Ain intermediate and the path CV endpoints.

Background

The JACS 2019 study uses three PDB structures:

> **What is a residue?** A residue is one building block of a protein chain. In proteins, each residue is one amino acid (like alanine, glycine, lysine, etc.). Non-standard residues (like our PLP-lysine) need special treatment.

PDB	Role	Residue of Interest	Source
5DVZ	Ain (internal aldimine) parameterization	LLP (24 heavy atoms, chain A)	`[SI p.S2]`

PDB	Role	Residue of Interest	Source
1WDW	Open-state endpoint for path CV	–	[SI p.S3]
3CEP	Closed-state endpoint for path CV	–	[SI p.S3]

PLP vs LLP — what is the difference?

This is a common source of confusion. Here is the short version:

- **PLP** (pyridoxal 5'-phosphate) is the **free cofactor** — the small molecule by itself, not attached to anything. Think of it as a key sitting on a table.
- **LLP** is what the PDB database calls PLP **after it has been covalently bonded to a lysine residue** (Lys82 in our case) through a Schiff base linkage (a C=N bond). (A Schiff base is a chemical bond – specifically a C=N double bond – formed when an aldehyde reacts with an amine. In TrpB, PLP's aldehyde group reacts with Lys82's amino group to form this bond, covalently linking the cofactor to the protein.) Think of it as the key inserted into a lock — it is chemically different from the free key because a new bond has formed.

In the crystal structure 5DVZ, PLP is not floating freely — it is covalently attached to Lys82. So the PDB registers it as **LLP** (full name: N6-(pyridoxal phosphate)-L-lysine), not PLP. If you search for "PLP" in the 5DVZ PDB file, you will find nothing. The residue name is **LLP**.

Throughout this tutorial: - We say "PLP" when talking about the cofactor **in general** (e.g., "PLP parameterization") - We use "LLP" when referring to the **specific PDB residue** in our files - Later, antechamber renames it to "AIN" (our shorthand for the Ain intermediate) — this is just a label change and does not affect the chemistry

Bottom line: PLP, LLP, and AIN all refer to the same molecule in different contexts. The atoms and charges are identical.

Step 1.1: Download PDB files

```
cd $WORK/structures

wget -O 5DVZ.pdb https://files.rcsb.org/download/5DVZ.pdb
wget -O 1WDW.pdb https://files.rcsb.org/download/1WDW.pdb
wget -O 3CEP.pdb https://files.rcsb.org/download/3CEP.pdb
```

```
# Verify downloads succeeded:
ls -lh $WORK/structures/*.pdb
# Expected: three files, each > 100 KB
# If any file is 0 bytes or contains "404 Not Found":
#   - Check the URL is correct
```

```
#   - Try: curl -L <url> -o <filename>
#   - Check your internet connection
```

Verification:

> **What is `grep`?** `grep` searches for text patterns inside files or command output. `grep "hello" file.txt` finds all lines containing "hello". The `-c` flag counts matching lines instead of showing them. The `-q` flag silently checks if a match exists (used in scripts).

```
ls -lh $WORK/structures/*.pdb
# Should show three PDB files, each > 100 KB

# Confirm LLP residue in 5DVZ:
grep "^HETATM" $WORK/structures/5DVZ.pdb \
  | awk '{print substr($0,18,3)}' | sort -u
# Should include: LLP
```

> **What is `sort -u`?** `sort` arranges lines alphabetically. The `-u` flag keeps only unique lines (removes duplicates). Together, `sort -u` gives you a deduplicated list.
>
> **What is `awk`?** `awk` is a text-processing tool that works column by column. In a PDB file, each line has fixed-width columns (atom name at positions 13-16, residue name at 18-20, chain ID at position 22, etc.). `awk` lets us extract specific columns. For example, `substr($0,18,3)` extracts 3 characters starting at position 18 – that's the residue name in PDB format.
>
> **What is `gsub` in `awk`?** `gsub(/pattern/, "replacement", variable)` does a global substitution – it replaces all occurrences of `pattern` with `replacement` in the variable. `gsub(/ /,"",rn)` removes all spaces from the variable `rn`.
>
> **What is `grep -E`?** The `-E` flag enables "extended" regular expressions. The pattern `^(ATOM  |HETATM)` means "lines starting with either ATOM or HETATM" – these are the two record types in PDB files that contain atom coordinates. The `^` means "start of line", and `|` means "or".

```
# Count LLP heavy atoms in chain A:
grep -E "^(HETATM|ATOM  )" \
  $WORK/structures/5DVZ.pdb \
  | awk '{rn=substr($0,18,3); gsub(/ /,"",rn);
         cid=substr($0,22,1);
         if(rn=="LLP" && cid=="A") print}' \
  | wc -l
# Expected: 24
```

> **What is `wc`?** `wc` stands for "word count". `wc -l` counts the number of lines in a file or in command output. We use it to verify that files have the expected number of entries.

Step 1.2: Extract chain A from 5DVZ

> **Why**: 5DVZ has 4 chains (A, B, C, D). We only need chain A for the benchmark system (standalone beta subunit). `[SI p.S2]`

Understanding PDB file format: A PDB file stores 3D coordinates of every atom in a protein. Each line represents one atom and has a fixed-column format: - Columns 1-6: Record type (`ATOM` for protein atoms, `HETATM` for ligands/cofactors) - Columns 13-16: Atom name (e.g., `CA` for alpha carbon, `NZ` for nitrogen zeta) - Columns 18-20: Residue name (e.g., `ALA` for alanine, `LLP` for PLP-lysine) - Column 22: **Chain ID** – a single letter identifying which protein chain this atom belongs to - Columns 23-26: Residue number - Columns 31-54: X, Y, Z coordinates in angstroms

Why Chain A? Crystal structures often contain multiple copies of the same protein in the unit cell. PDB 5DVZ has 4 chains (A, B, C, D), all identical copies of TrpB. We pick Chain A arbitrarily – any chain would give the same result. The chain ID is at column 22 of each ATOM/HETATM line.

```
grep -E "^(ATOM  |HETATM)" $WORK/structures/5DVZ.pdb \
  | awk '{cid=substr($0,22,1); if(cid=="A") print}' \
  > $WORK/structures/5DVZ_chainA.pdb
```

Verification:

```
wc -l $WORK/structures/5DVZ_chainA.pdb
# Should be ~2800-3200 lines (one chain of a ~390-residue protein + LLP)

grep "LLP" $WORK/structures/5DVZ_chainA.pdb | wc -l
# Expected: 24
```

Step 1.3: Prepare the protein PDB for tleap

What: Rename the LLP residue to AIN (our internal name for the Ain intermediate) and clean up the PDB for AMBER processing.

Why: antechamber outputs a mol2 file with residue name "AIN" (derived from the output filename, see FP-017). We must make the PDB and mol2 residue names match for tleap to recognize the cofactor.

The naming flow: PDB file has "LLP" → we rename to "AIN" in the PDB → antechamber also outputs "AIN" in the mol2 → both files match → tleap can link them together. If the names don't match, tleap won't recognize the cofactor and will fail.

What is sed? `sed` (stream editor) finds and replaces text in files. `sed 's/old/new/g' file` replaces every occurrence of "old" with "new". Here we use it to rename the residue from LLP to AIN throughout the file.

What is `grep -v`? The `-v` flag **inverts** the match – it shows lines that do NOT contain the pattern. So `grep -v "HOH"` removes all lines containing "HOH" (water molecules). Think of `-v` as "exclude". We chain two `grep -v` commands to remove waters AND alternate conformations.

```
cd $WORK/structures

# Remove waters, ions, and alternate conformations; rename LLP -> AIN
grep -E "^(ATOM  |HETATM)" 5DVZ_chainA.pdb \
```

```
  | grep -v "HOH" \
  | grep -v " B " \
  | sed 's/LLP/AIN/g' \
  > $WORK/systems/pftrps_ain/5DVZ_chainA_prepared.pdb
```

Verification:

```
grep "AIN" $WORK/systems/pftrps_ain/5DVZ_chainA_prepared.pdb | wc -l
# Expected: 24 (the renamed LLP atoms)

grep "LLP" $WORK/systems/pftrps_ain/5DVZ_chainA_prepared.pdb | wc -l
# Expected: 0 (all renamed)
```

Phase 2: PLP Cofactor Parameterization (~2–3 days)

Goal: Generate AMBER-compatible force field parameters (GAFF atom types + RESP charges) for the PLP-K82 Schiff base cofactor (Ain intermediate).

What are RESP charges? RESP (Restrained Electrostatic Potential) fitting assigns partial charges to each atom by matching to a quantum mechanical calculation. These charges determine how strongly each atom attracts or repels its neighbors.

What is a force field? A force field is a set of mathematical equations and parameters that describe how atoms interact – how strongly bonds stretch, how angles bend, how charged atoms attract or repel each other. ff14SB is one such parameter set, designed specifically for proteins.

Why this is necessary: The ff14SB force field only contains parameters for the 20 standard amino acids. LLP (pyridoxal phosphate covalently bound to Lys82 via a Schiff base) is a non-standard residue that must be parameterized separately. The SI specifies GAFF atom types with RESP charges fitted at the HF/6-31G(d) level [SI p.S2].

This is the most complex phase of the entire pipeline. Each substep is explained in detail below.

Overview of the RESP Parameterization Workflow

```
PDB (5DVZ, LLP residue)
  |
  v
[Step 2.1] Extract LLP fragment from PDB
  |         Purpose: isolate the non-standard residue
  v
[Step 2.2] Add hydrogens (reduce)
  |         Purpose: antechamber needs H to determine bond orders
  v
[Step 2.3] Add ACE/NME capping groups
  |         Purpose: neutralize dangling peptide bonds for QM
```

```
  v
[Step 2.4] Generate Gaussian input (antechamber)
  |         Purpose: set up QM geometry optimization + ESP calculation
  v
[Step 2.5] Run Gaussian 16 QM calculation (~12-48 hrs)
  |         Purpose: compute electrostatic potential around molecule
  v
[Step 2.6] Extract RESP charges (antechamber)
  |         Purpose: fit point charges to reproduce QM ESP
  v
[Step 2.7] Generate missing parameters (parmchk2)
  |         Purpose: estimate bonded terms not in GAFF
  v
[Step 2.8] Validate output files
          Purpose: ensure charges, atom types, and connectivity are correct
```

Step 2.1: Extract the LLP fragment from 5DVZ

What: Pull out the 24 heavy atoms of the LLP residue from chain A of 5DVZ.

Why: We need an isolated fragment of the non-standard residue for quantum mechanical charge calculation. The rest of the protein will use ff14SB parameters.

```
cd $WORK/parameters/plp_structures

grep -E "^(HETATM|ATOM  )" $WORK/structures/5DVZ.pdb \
  | awk '{rn=substr($0,18,3); gsub(/ /,"",rn);
         cid=substr($0,22,1);
         if(rn=="LLP" && cid=="A") print}' \
  > $WORK/parameters/plp_structures/LLP_chainA.pdb
```

Verification:

```
wc -l $WORK/parameters/plp_structures/LLP_chainA.pdb
# Expected: 24 lines (24 heavy atoms)

# Print atom names to verify completeness:
awk '{printf "%s\n", substr($0,13,4)}' \
  $WORK/parameters/plp_structures/LLP_chainA.pdb
# Should include backbone atoms (N, CA, C, O)
# and PLP ring atoms (C1, C2, C3, etc.)
```

Step 2.2: Add hydrogens with reduce

What: Use the `reduce` program (bundled with AmberTools) to add hydrogen atoms to the extracted LLP fragment.

Why: antechamber requires hydrogen atoms to correctly determine bond orders and aromaticity. Without hydrogens, the pyridine ring atoms are misassigned as `c1` (sp)

instead of ca (aromatic), producing incorrect force field parameters. This was documented as failure pattern FP-010.

Purpose of reduce: reduce uses standard bond lengths, van der Waals radii, and hydrogen bonding patterns to place hydrogens at chemically reasonable positions. It handles: (1) backbone amide NH, (2) aromatic CH, (3) methyl CH3, (4) hydroxyl OH.

```
module load amber/24p3

reduce $WORK/parameters/plp_structures/LLP_chainA.pdb \
  > $WORK/parameters/plp_structures/LLP_chainA_H.pdb
```

Verification:

```
wc -l $WORK/parameters/plp_structures/LLP_chainA_H.pdb
# Expected: ~42 lines (24 heavy atoms + ~18 hydrogens)

# Count hydrogens:
grep " H" $WORK/parameters/plp_structures/LLP_chainA_H.pdb | wc -l
# Expected: ~18 hydrogen atoms
```

Step 2.3: Add ACE/NME capping groups

What: Attach acetyl (ACE) and N-methylamine (NME) capping groups to the backbone N-terminus and C-terminus of the LLP fragment.

Why: Cutting LLP out of the protein leaves dangling peptide bonds at the backbone N and C atoms. In reality, these atoms are bonded to neighboring residues (Gly81 and Ser83). Without capping, the QM calculation would see unphysical open valences, producing distorted charges at the termini. The caps simulate the electronic environment of the protein backbone.

ACE cap (CH3-CO-): Mimics the carbonyl of the preceding residue. Placed bonded to backbone N. **NME cap** (NH-CH3): Mimics the amide of the following residue. Placed bonded to backbone C.

Note: This step requires manual intervention. The recommended approach is to use a molecular editor (PyMOL Builder or UCSF Chimera) to add the caps and save the result. Alternatively, use the Python script build_llp_ain_capped_resp.py which was developed and validated for this project.

```
# The capped structure should have:
#   - 17 carbon atoms
#   - 4 nitrogen atoms
#   - 7 oxygen atoms
#   - 1 phosphorus atom
#   - 25 hydrogen atoms
#   Total: 54 atoms
#
# Z_total (sum of atomic numbers) = 226 (MUST be even)
# Net charge = -2   [Caulkins 2014]
```

```
# Electrons = 226 - (-2) = 228 (even -> singlet OK)
#
# FP-012 guard: (Z_total - charge) must be even for singlet multiplicity
```

After capping (by whatever method), save the result:

```
ls -lh $WORK/parameters/plp_structures/LLP_ain_capped.pdb
# Expected: file exists, ~54 atoms
```

Verification:

```
# Count total atoms in capped structure:
grep -c "^ATOM\|^HETATM" $WORK/parameters/plp_structures/LLP_ain_capped.pdb
# Expected: 54

# Verify electron count parity (CRITICAL, see FP-012):
# Z_total = 6*17 + 7*4 + 8*7 + 15*1 + 1*25 = 102 + 28 + 56 + 15 + 25 = 226
# Electrons = 226 - (-2) = 228 (even) -> singlet (mult=1) is valid
```

Step 2.4: Prepare Gaussian input for RESP ESP calculation

What: Create a Gaussian 16 input file that performs (1) geometry optimization and (2) Merz-Kollman electrostatic potential (ESP) calculation on the capped LLP fragment.

Why: RESP charges are derived by fitting point charges to reproduce the quantum mechanical electrostatic potential around the molecule. The HF/6-31G(d) level is the standard for RESP fitting because it systematically overestimates dipole moments by ~10-20%, which partially compensates for the lack of explicit polarization in fixed-charge force fields like GAFF. `[SI p.S2]`

Why RESP and not AM1-BCC? AM1-BCC is faster but uses a semi-empirical method that fails for PLP – the SCF (self-consistent field) calculation doesn't converge due to PLP's complex electronic structure (conjugated aromatic ring + phosphate + imine). RESP uses a full quantum mechanical calculation (HF/6-31G(d)) which handles PLP correctly, at the cost of needing Gaussian. This is documented as FP-009.

The Gaussian route line explained:

Keyword	Purpose
`HF/6-31G*`	Hartree-Fock theory with 6-31G(d) basis set. Standard for RESP. `[SI p.S2]`
`SCF=tight`	Tighter SCF convergence criteria (10^-8 a.u.). Ensures reliable ESP.
`Pop=MK`	Merz-Kollman population analysis: outputs ESP on a grid of points around the molecule. This is what RESP fitting reads.
`iop(6/33=2)`	Sets the number of concentric layers of ESP grid points to 2. More layers = better charge fitting.

Keyword	Purpose
`iop(6/42=6)`	Sets the density of points per unit area on each layer. Higher = more ESP data points.
`opt`	Geometry optimization: relaxes the capped structure to a local minimum at the HF/6-31G(d) level before computing ESP.

> **Note on iop(6/50=1)**: Some RESP tutorials include `iop(6/50=1)` to write a separate `.gesp` file. We do NOT use this because it causes "Blank file name read" errors in Gaussian 16c02 (documented as FP-013). Instead, antechamber reads the ESP data directly from the Gaussian `.log` file using the `-fi gout` flag (see FP-014).

Net charge decomposition (Ain/LLP)

> **Source**: Caulkins et al.[4] (solid-state NMR of PLP Schiff bases)

Functional Group	Charge	Rationale
Phosphate group (PO4)	-2	Three deprotonated oxygens at physiological pH
Pyridinium N1	+1	Protonated ring nitrogen (pKa ~5.5, protonated at pH 7.8)
Phenolate O3	-1	Deprotonated phenol oxygen (pKa ~3-4)
Schiff base NZ (imine)	0	Neutral imine linkage in Ain state
ACE/NME caps	0	Charge-neutral by design
Net	**-2**	-2 + 1 + (-1) + 0 + 0 = **-2**

```
cat > $WORK/parameters/resp_charges/LLP_ain_resp_capped.gcrt << 'EOF'
%chk=LLP_ain_resp_capped.chk
%nprocshared=8
%mem=8GB
#HF/6-31G* SCF=tight Pop=MK iop(6/33=2) iop(6/42=6) opt

ACE-LLP-NME (Ain) RESP; charge=-2 [Caulkins 2014]

-2 1
[COORDINATES GO HERE]
[Paste Cartesian coordinates from the capped PDB, one atom per line]
[Format: Element   X.xxxxxx   Y.yyyyyy   Z.zzzzzz]
```

[4] Caulkins, B. G.; Bastin, B.; Yang, C.; Neubauer, T. J.; Young, R. P.; Hilario, E.; Huang, Y.-m. M.; Chang, C.-e. A.; Fan, L.; Dunn, M. F.; Marsella, M. J.; Mueller, L. J. *J. Am. Chem. Soc.* **2014**, *136*, 12824-12827.

```
EOF
```

Generate the Gaussian input file

> **In practice**, the coordinates are generated by the `build_llp_ain_capped_resp.py` script which reads the capped PDB and formats them correctly. The script also validates the electron count parity (FP-012 guard).

Verification:

```
head -6 $WORK/parameters/resp_charges/LLP_ain_resp_capped.gcrt
# Should show: %chk, %nprocshared=8, %mem=8GB,
# #HF/6-31G* route line, title, charge/mult

grep -c "^[A-Z]" $WORK/parameters/resp_charges/LLP_ain_resp_capped.gcrt
# Expected: ~54 (one line per atom in the coordinate block)
```

Step 2.5: Submit Gaussian QM calculation

> **What**: Run the Gaussian 16 geometry optimization + ESP calculation on Longleaf.
>
> **Why**: This is the computational bottleneck of parameterization. HF/6-31G(d) on a 54-atom system typically takes 12–48 hours on 8 cores.
>
> **Expected output**: A `.log` file containing the optimized geometry and Merz-Kollman ESP data. Look for "Normal termination of Gaussian 16" at the end.
>
> **Expected runtime**: Gaussian geometry optimization + ESP on a ~54-atom capped fragment at HF/6-31G(d): typically 12-48 hours on 8 CPU cores. If it takes longer than 3 days, check for SCF convergence issues in the log file.

```
cat > $WORK/scripts/submit_gaussian_capped.slurm << 'EOF'
#!/bin/bash
#SBATCH --job-name=ain_resp_qm
#SBATCH --partition=general
#SBATCH --nodes=1
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=8
#SBATCH --mem=16G
#SBATCH --time=48:00:00
#SBATCH --output=$WORK/logs/gaussian_ain_resp_%j.out

# Load Gaussian
module load gaussian/16c02

# Set scratch directory (Gaussian writes large temp files)
export GAUSS_SCRDIR=$WORK/logs/gaussian_scratch_${SLURM_JOB_ID}
mkdir -p ${GAUSS_SCRDIR}
```

```
cd $WORK/parameters/resp_charges/

echo "Starting Gaussian at $(date)"
g16 LLP_ain_resp_capped.gcrt
echo "Gaussian finished at $(date)"

# Clean up scratch
rm -rf ${GAUSS_SCRDIR}
EOF

sbatch $WORK/scripts/submit_gaussian_capped.slurm
```

> **What is `sbatch`?** On an HPC cluster, you don't run long computations directly – you submit them to a job scheduler (SLURM). `sbatch script.sh` submits your script to a queue. The scheduler finds an available compute node with the resources you requested (GPUs, memory, etc.) and runs your job there. You can check its status with `squeue -u $USER`.

Verification (after job completes):

```
# Check for successful termination:
tail -5 $WORK/parameters/resp_charges/LLP_ain_resp_capped.log
# Must contain: "Normal termination of Gaussian 16"

# Check that ESP data was written (Merz-Kollman charges section):
grep -c "Merz-Kollman" $WORK/parameters/resp_charges/LLP_ain_resp_capped.log
# Expected: >= 1

# Check for SCF convergence issues:
grep "Convergence failure" $WORK/parameters/resp_charges/LLP_ain_resp_capped.log
# Expected: (no output = good)
```

Step 2.6: Extract RESP charges with antechamber

> **What**: Use antechamber to read the Gaussian log file and fit RESP point charges to reproduce the QM electrostatic potential. Simultaneously assign GAFF atom types.
>
> **Why**: RESP (Restrained Electrostatic Potential) fitting produces atomic partial charges that are compatible with the AMBER force field family. The fitting procedure: 1. Reads the Merz-Kollman ESP grid points from the Gaussian log 2. Fits point charges on each atom to minimize the difference between QM ESP and the ESP produced by the point charges 3. Applies hyperbolic restraints to non-hydrogen atoms (pushes charges toward zero to avoid overfitting) 4. Uses two-stage fitting: first stage restraints are weak (0.0005), second stage adds stronger restraints (0.001) on buried atoms
>
> **Key antechamber flags explained**: - `-fi gout`: Read Gaussian output format (log file). This extracts both the optimized geometry and the ESP data. We use this instead of `-fi gesp` because we do not generate a separate .gesp file (see FP-013/FP-014). - `-fo mol2`: Output in Tripos mol2 format (contains atom types, charges, and connec-

tivity) - `-c resp`: Use RESP charge fitting method [SI p.S2] - `-at gaff`: Assign GAFF (General AMBER Force Field) atom types [SI p.S2] - `-nc -2`: Net charge of the molecule is -2 [Caulkins 2014] - `-m 1`: Multiplicity is 1 (singlet, all electrons paired) - `-rn AIN`: Set residue name to AIN (note: antechamber may override this with the output filename, see FP-017)

```
module load amber/24p3

cd $WORK/parameters

antechamber \
  -i resp_charges/LLP_ain_resp_capped.log \
  -fi gout \
  -o mol2/Ain_gaff.mol2 \
  -fo mol2 \
  -at gaff \
  -c resp \
  -nc -2 \
  -m 1 \
  -rn "AIN" \
  2>&1 | tee $WORK/logs/antechamber_resp_ain.log
```

Verification:

```
ls -lh $WORK/parameters/mol2/Ain_gaff.mol2
# Should exist, ~3-5 KB

# Check total charge sums to -2:
grep "@<TRIPOS>ATOM" -A 100 \
  $WORK/parameters/mol2/Ain_gaff.mol2 \
  | awk 'NF==9 {sum+=$9} END {
      printf "Total charge: %.4f\n", sum}'
# Expected: -2.0000 (within rounding)

# Check atom types are aromatic for pyridine ring:
grep -E "\bca\b|\bnb\b" $WORK/parameters/mol2/Ain_gaff.mol2
# Should show ca (aromatic C) and nb (aromatic N) for pyridine atoms

# Verify residue name in mol2 (FP-017 check):
grep "SUBSTRUCTURE" -A 1 $WORK/parameters/mol2/Ain_gaff.mol2
# Will show "AIN" (from filename, not from -rn flag)
```

Step 2.7: Generate missing bonded parameters with parmchk2

What is a dihedral angle? A dihedral angle is defined by four atoms (A-B-C-D) and describes the rotation around the B-C bond. It controls the 3D shape of the molecule.

What: Use parmchk2 to identify any bond, angle, dihedral, or improper torsion terms in the Ain mol2 that are not covered by the standard GAFF parameter database, and

estimate them from similar terms.

Why: GAFF covers most common organic molecular fragments, but the PLP phosphate group (P-O bonds) and Schiff base linkage (C=N-C) have unusual combinations of GAFF atom types. parmchk2 searches the GAFF database for the closest matching parameters and extrapolates. Terms with high penalty scores (> 10) should be reviewed manually.

```
parmchk2 \
  -i $WORK/parameters/mol2/Ain_gaff.mol2 \
  -f mol2 \
  -o $WORK/parameters/frcmod/Ain.frcmod
```

Verification:

```
ls -lh $WORK/parameters/frcmod/Ain.frcmod
# Should exist, ~1-3 KB

# Check for ATTN warnings (parameters estimated with high penalty):
grep -c "ATTN" $WORK/parameters/frcmod/Ain.frcmod
# Ideal: 0. If > 0, review the terms:
grep "ATTN" $WORK/parameters/frcmod/Ain.frcmod
```

Our result: 0 ATTN warnings (validated 2026-03-31, skeptic 6/6 PASS).

Step 2.8: Post-processing – retype backbone atoms

What: Replace GAFF atom types for the K82 backbone atoms (C, CA, N, O, CB, CG, CD, CE) with their ff14SB equivalents, so that tleap can form correct peptide bonds with neighboring residues.

Why: antechamber assigns GAFF types to ALL atoms, including the protein backbone (e.g., c3 for CA instead of ff14SB's CT). When tleap builds the full protein, it needs the backbone atoms to have ff14SB types to match the protein force field. Only the PLP-specific atoms (pyridine ring, phosphate, Schiff base) should keep their GAFF types.

Atoms to retype: C8 (backbone C alpha) -> CT, backbone C -> C, backbone N -> N, backbone O -> O, sidechain CB/CG/CD/CE -> CT, sidechain H -> HC/HP

This is done by editing the mol2 file. The specific atom name-to-type mappings depend on your exact mol2 output. See the validated result:

```
# Our validated mol2 has 42 atoms with:
# - Backbone retyped to ff14SB: C8(CA)->CT, C(C)->C, N->N, O->O
# - Sidechain retyped: CB,CG,CD,CE -> CT (ff14SB lysine sidechain types)
# - NZ stays GAFF 'nh' (connects to PLP ring)
# - All PLP-specific atoms keep GAFF types (ca, nb, oh, os, p5, o, etc.)
```

Verification:

```
# Final atom count in mol2:
grep -c "^[[:space:]]*[0-9]" $WORK/parameters/mol2/Ain_gaff.mol2
```

```
# Expected: 42 (including hydrogens on the
# capped fragment minus cap atoms that are removed)

# Total charge:
grep "@<TRIPOS>ATOM" -A 100 \
  $WORK/parameters/mol2/Ain_gaff.mol2 \
  | awk 'NF==9 {sum+=$9} END {
      printf "Total charge: %.4f\n", sum}'
# Expected: -2.0000
```

Phase 2 Summary

Output File	Contents	Status
`parameters/mol2/Ain_gaff.mol2`	GAFF atom types + RESP charges, 42 atoms, charge=-2	Validated
`parameters/frcmod/Ain.frcmod`	Missing bonded parameters (0 ATTN warnings)	Validated
`parameters/resp_charges/LLP_ain_gas_ap.log`	Gaussian 16 QM output (Normal termination)	Validated

Phase 3: Build System with tleap (~5 min)

Goal: Combine the parameterized Ain cofactor with the protein, solvate in a TIP3P water box, and neutralize with counterions.

What is a topology? A topology file is like a blueprint of your molecular system. It lists every atom, which atoms are bonded together, and what force field parameters govern their interactions (spring constants for bonds, charge values, etc.).

What tleap does: tleap is AMBER's system builder. It reads force field libraries (ff14SB for protein, GAFF for cofactor, TIP3P for water), loads the prepared PDB, solvates it in a periodic box, adds counterions, and writes the topology (.parm7) and coordinate (.inpcrd) files needed for MD.

Step 3.1: Create tleap library file from mol2

Why: tleap needs a `.lib` (or `.off`) file to recognize the AIN residue. We convert our validated mol2 to this format.

```
cat > $WORK/scripts/tleap_pftrps_ain.in << 'EOF'
# tleap input for building PfTrpS(Ain) system
# Reference: JACS 2019 SI p.S2
# (ff14SB + GAFF + TIP3P + 10 A box + Na+ neutralization)
```

```
# Load force fields
source leaprc.protein.ff14SB      # Protein force field [SI p.S2]
source leaprc.gaff                 # GAFF for PLP cofactor [SI p.S2]
source leaprc.water.tip3p          # TIP3P water model [SI p.S2]

# Load Ain (PLP-K82 Schiff base) parameters
AIN = loadmol2 ../parameters/mol2/Ain_gaff.mol2
set AIN head AIN.1.N               # N-terminal connection point (backbone N)
set AIN tail AIN.1.C               # C-terminal connection point (backbone C)
set AIN.1 connect0 AIN.1.N  # preceding residue connection
set AIN.1 connect1 AIN.1.C  # following residue connection
saveoff AIN ../systems/pftrps_ain/Ain_gaff.lib  # Save as library for reuse

# Load Ain frcmod (missing GAFF parameters)
loadamberparams ../parameters/frcmod/Ain.frcmod

# Reload library (ensures it's registered)
loadoff ../systems/pftrps_ain/Ain_gaff.lib

# Load the prepared protein PDB
# The PDB has LLP renamed to AIN, matching the mol2/lib residue name
prot = loadpdb ../systems/pftrps_ain/5DVZ_chainA_prepared.pdb
check prot

# Solvate in cubic TIP3P box with 10 A buffer [SI p.S2]
solvatebox prot TIP3PBOX 10.0

# Neutralize with Na+ counterions [SI p.S2]
# (see "counterions" note below)
addions prot Na+ 0

# Final check
check prot

# Save AMBER topology and coordinates
saveamberparm prot \
  ../systems/pftrps_ain/pftrps_ain.parm7 \
  ../systems/pftrps_ain/pftrps_ain.inpcrd
savepdb prot ../systems/pftrps_ain/pftrps_ain_leap.pdb
quit
EOF
```

What are counterions? Proteins usually have a net electric charge. Counterions (Na+ or Cl-) are added to neutralize this charge. Without them, the simulation would have unphysical long-range electrostatic effects.

Step 3.2: Run tleap

```
module load amber/24p3

cd $WORK/scripts
tleap -f tleap_pftrps_ain.in 2>&1 | tee $WORK/logs/tleap_pftrps_ain.log
```

> **If this fails:** If tleap reports FATAL errors, check `$WORK/logs/tleap_pftrps_ain.log` for details. Common issues: (1) missing mol2/frcmod files – verify paths are correct relative to the tleap input file; (2) wrong residue names – the PDB must use "AIN" matching the mol2; (3) "Could not find unit" – make sure `loadoff` and `loadamberparams` come before `loadpdb`.

Verification:

```
# Check output files exist and have reasonable sizes:
ls -lh $WORK/systems/pftrps_ain/pftrps_ain.parm7
# Expected: ~600 KB - 1 MB

ls -lh $WORK/systems/pftrps_ain/pftrps_ain.inpcrd
# Expected: ~1-2 MB

# Check total atom count:
grep -c "^ATOM\|^HETATM" $WORK/systems/pftrps_ain/pftrps_ain_leap.pdb
# Expected: ~39,000 (protein ~6,000 + waters ~33,000)

# Check tleap log for errors:
grep -i "error\|fatal\|warning" $WORK/logs/tleap_pftrps_ain.log
# Review any warnings. Common benign warnings:
# "close contact" between newly added atoms.

# Check neutralization:
grep "Na+" $WORK/logs/tleap_pftrps_ain.log
# Should show 4 Na+ ions added
# (our system has net charge -4 before neutralization)

# Check box dimensions:
grep "Box dimensions" $WORK/logs/tleap_pftrps_ain.log
# Expected: approximately 76 x 88 x 73 A
```

> **Our result**: 39,268 total atoms, box 76.4 x 88.1 x 73.2 A, 4 Na+ ions, net charge 0. SI reports ~15,000 water molecules; we have 11,092 (slightly smaller because we use only the beta subunit for the benchmark). `[SI p.S2]`

Phase 4: Classical MD – Prep Pipeline (~24 hrs)

Parameter reference: Every parameter in the AMBER input files below is explained in detail in Appendix A at the end of this document. If you encounter an unfamiliar parameter, check there first.

Goal: Prepare the solvated system for production MD through minimization, staged heating, and equilibration.

What is equilibration? After minimization, the system has low energy but isn't yet behaving like a real physical system at a given temperature and pressure. Equilibration runs a short simulation with controlled temperature/pressure until properties like density and energy stabilize.

What is NVT? NVT means the Number of atoms, Volume, and Temperature are held constant. The simulation box size doesn't change, and a thermostat maintains the target temperature.

What is NPT? NPT means the Number of atoms, Pressure, and Temperature are held constant. The box can expand or shrink to maintain the target pressure (usually 1 atm), which is how the system finds the correct density.

Protocol from SI `[SI pp.S2–S3]`: 1. Two-stage minimization (restrained + unrestrained) 2. Seven-step heating (0 -> 350 K in 50 K increments, NVT, 50 ps each) 3. NPT equilibration (2 ns at 350 K, 1 atm)

What is energy minimization? Energy minimization adjusts atom positions to reduce the system's potential energy. Imagine gently shaking a box of marbles until they settle into the lowest spots – that's what minimization does to atoms. It fixes bad contacts (atoms too close together) from the initial structure.

Step 4.1: Create minimization input files

min1.in – Restrained minimization

What: Minimize the system energy while holding protein heavy atoms fixed.

Why: After solvation, water molecules and ions may have unfavorable contacts with the protein surface. Restrained minimization relaxes these contacts while preserving the crystal structure geometry. The 500 kcal/mol/A^2 restraint is very strong – it essentially freezes the protein. `[SI p.S2]`

```
cat > $WORK/runs/pftrps_ain_md/min1.in << 'EOF'
PfTrpS(Ain) restrained min; JACS 2019 SI Min Stage 1
&cntrl
  imin=1,
  ntx=1,
  irest=0,
  maxcyc=10000,
  ncyc=5000,
  ntpr=500,
  ntb=1,
```

```
  cut=8.0,
  ntr=1,
  restraint_wt=500.0,
  restraintmask='!@H= & !:WAT & !:Na+',
/
EOF
```

Parameter	Value	Meaning	Source
`imin=1`	1	Run minimization (not MD)	AMBER manual
`ntx=1`	1	Read coordinates only (no velocities needed for minimization)	AMBER manual
`irest=0`	0	Fresh start (not a restart)	AMBER manual
`maxcyc=10000`	10000	Maximum 10,000 minimization cycles	`[operational default]`
`ncyc=5000`	5000	First 5,000 cycles use steepest descent (robust), then switch to conjugate gradient (faster convergence)	`[operational default]`
`ntpr=500`	500	Print energy every 500 steps (monitor convergence)	`[operational default]`
`ntb=1`	1	Periodic boundary conditions, constant volume	AMBER manual
`cut=8.0`	8.0	Non-bonded interaction cutoff at 8 A	`[SI p.S3]`
`ntr=1`	1	Apply positional restraints	`[SI p.S2]`
`restraint_wt=500.0`	500.0	Restraint force constant: 500 kcal/mol/A^2 (very strong)	`[SI p.S2]`
`restraintmask`	`'!@H= & !:WAT & !:Na+'`	Restrain all non-hydrogen, non-water, non-ion atoms (i.e., protein + cofactor heavy atoms)	`[SI p.S2]`

What are periodic boundary conditions? The simulation box is surrounded by infinite copies of itself. When an atom exits one side, it re-enters from the opposite side. This trick avoids edge effects and simulates a small piece of a much larger system.

Understanding AMBER atom masks: The `restraintmask` uses a special syntax to

select atoms: - `@` selects by atom name/type, `:` selects by residue name - `!` means NOT (exclude), & means AND - @H= matches all hydrogen atoms (the = matches by element) - So `'!@H= & !:WAT & !:Na+'` means "everything that is NOT hydrogen AND NOT water AND NOT sodium ions" – i.e., all heavy atoms of the protein and cofactor

min2.in – Unrestrained minimization

What: Minimize the entire system without restraints.

Why: After the water shell is relaxed (min1), release all restraints and let the full system find a local energy minimum. This resolves any remaining steric clashes. `[SI p.S2]`

```
cat > $WORK/runs/pftrps_ain_md/min2.in << 'EOF'
PfTrpS(Ain) unrestrained min; JACS 2019 SI Min Stage 2
&cntrl
  imin=1,
  ntx=1,
  irest=0,
  maxcyc=10000,
  ncyc=5000,
  ntpr=500,
  ntb=1,
  cut=8.0,
  ntr=0,
/
EOF
```

Step 4.2: Create heating input files (7 steps, 0 -> 350 K)

What: Gradually heat the system from 0 K to 350 K in seven 50 ps NVT steps, with progressively decreasing positional restraints on protein heavy atoms.

Why seven heating steps instead of one? Jumping from 0 K to 350 K instantly would assign huge random velocities to atoms, causing violent collisions and potentially crashing the simulation. Gradual heating (50 K increments) lets the system relax at each temperature before moving higher.

Why: Sudden heating causes instability (atoms fly apart). Gradual heating with restraints allows the system to thermalize safely. The restraint schedule from SI `[SI p.S2]` starts at 210 kcal/mol/A^2 (strong) and decreases to 10 kcal/mol/A^2 (weak), letting the protein gradually relax as thermal energy increases.

Temperature 350 K: *P. furiosus* is a hyperthermophilic archaeon (optimal growth at 100 C). The enzyme functions at high temperature; 350 K (77 C) is the simulation temperature from SI. `[SI p.S3]`

Why do restraints decrease over heating steps? Strong restraints (210 kcal/mol/A^2) at low temperature prevent the protein from distorting. As temperature increases and the system stabilizes, restraints weaken (210 -> 165 -> 125 -> 85 -> 45 -> 10 -> 10) to allow natural protein flexibility.

Note on restraint schedule: The SI lists 6 restraint values (210, 165, 125, 85, 45, 10) for 7 heating steps. We use 10.0 for both heat6 and heat7 as an operational choice. `[operational default]`

```
cat > $WORK/runs/pftrps_ain_md/heat1.in << 'EOF'
PfTrpS(Ain) heat1; SI Heating 0->50 K, 210 kcal/mol/A^2
&cntrl
  imin=0,
  irest=0,
  ntx=1,
  nstlim=50000,
  dt=0.001,
  ntc=2,
  ntf=2,
  ntt=3,
  gamma_ln=1.0,
  tempi=0.0,
  temp0=50.0,
  nmropt=1,
  ntb=1,
  ntp=0,
  cut=8.0,
  ntr=1,
  restraint_wt=210.0,
  restraintmask='!@H= & !:WAT & !:Na+',
  ntpr=5000,
  ntwx=5000,
  ntwe=5000,
  ntwr=50000,
  ioutfm=1,
/
&wt
  TYPE='TEMP0', ISTEP1=0, ISTEP2=50000, VALUE1=0.0, VALUE2=50.0,
/
&wt TYPE='END' /
EOF
```

heat1.in (0 -> 50 K)

Parameter	Value	Meaning	Source
`imin=0`	0	Run molecular dynamics (not minimization)	AMBER manual
`irest=0`	0	Fresh start (heat1 starts from minimized structure)	AMBER manual

Parameter	Value	Meaning	Source
ntx=1	1	Read coordinates only (no velocities from minimization)	AMBER manual
nstlim=50000	50000	50,000 steps x 0.001 ps = 50 ps per heating step	[SI p.S2]
dt=0.001	0.001	1 fs timestep (conservative, used during heating with restraints)	[operational default]
ntc=2	2	SHAKE constraints on bonds involving hydrogen	[SI p.S3]
ntf=2	2	Do not calculate forces for SHAKE-constrained bonds (saves compute)	[SI p.S3]
ntt=3	3	Langevin thermostat (stochastic dynamics)	[SI p.S3]
gamma_ln=1.0	1.0	Langevin collision frequency (ps^-1). Controls coupling strength.	[operational default]
tempi=0.0	0.0	Initial temperature (only in heat1; subsequent steps restart from previous)	AMBER manual
temp0=50.0	50.0	Target temperature for this step	[SI p.S2]
nmropt=1	1	Enable NMR-style restraints (here used for temperature ramping via &wt)	AMBER manual
ntb=1	1	Constant volume periodic boundaries (NVT ensemble)	[SI p.S2]
ntp=0	0	No pressure coupling (NVT)	[SI p.S2]
restraint_wt=210.0	210.0	Positional restraint: 210 kcal/mol/A^2	[SI p.S2]
ioutfm=1	1	Write trajectory in NetCDF format (binary, compressed)	[operational default]

Parameter	Value	Meaning	Source
`&wt TYPE='TEMP0'`	–	Ramp target temperature linearly from VALUE1 to VALUE2 over ISTEP1 to ISTEP2	AMBER manual

Why different timesteps? Heating steps use `dt=0.001` (1 fs) because strong restraints create sharp forces that need smaller steps to handle safely. Production uses `dt=0.002` (2 fs) because SHAKE constraints (ntc=2) allow larger steps when bonds to hydrogen are frozen.

What is SHAKE? SHAKE is an algorithm that freezes bond lengths involving hydrogen atoms at their equilibrium values. This allows us to use a larger timestep (2 fs instead of 1 fs), making simulations run twice as fast without sacrificing accuracy.

What is a thermostat? A thermostat controls the system's temperature by adding or removing kinetic energy. The Langevin thermostat (ntt=3) mimics random collisions with an invisible heat bath – like the protein is surrounded by a constant-temperature environment.

What is `nmropt=1`? This enables the `&wt` (weight) section that follows the main `&cntrl` block. The `&wt` section controls how parameters change over time – here, it ramps the temperature from VALUE1 to VALUE2 over ISTEP1 to ISTEP2 steps.

What is NetCDF? NetCDF is a compressed binary file format for storing trajectory data. You can't read it with a text editor, but analysis tools (cpptraj, MDAnalysis) read it directly.

```
cat > $WORK/runs/pftrps_ain_md/heat2.in << 'EOF'
PfTrpS(Ain) heat2; SI Heating 50->100 K, 165 kcal/mol/A^2
&cntrl
  imin=0,
  irest=1,
  ntx=5,
  nstlim=50000,
  dt=0.001,
  ntc=2,
  ntf=2,
  ntt=3,
  gamma_ln=1.0,
  temp0=100.0,
  nmropt=1,
  ntb=1,
  ntp=0,
  cut=8.0,
  ntr=1,
```

```
  restraint_wt=165.0,
  restraintmask='!@H= & !:WAT & !:Na+',
  ntpr=5000,
  ntwx=5000,
  ntwe=5000,
  ntwr=50000,
  ioutfm=1,
/
&wt
  TYPE='TEMP0', ISTEP1=0, ISTEP2=50000, VALUE1=50.0, VALUE2=100.0,
/
&wt TYPE='END' /
EOF
```

heat2.in (50 -> 100 K)

> **Key difference from heat1**: `irest=1` and `ntx=5` – this tells AMBER to restart from the previous step (read both coordinates AND velocities from the restart file).

```
cat > $WORK/runs/pftrps_ain_md/heat3.in << 'EOF'
PfTrpS(Ain) heat3; SI Heating 100->150 K, 125 kcal/mol/A^2
&cntrl
  imin=0,
  irest=1,
  ntx=5,
  nstlim=50000,
  dt=0.001,
  ntc=2,
  ntf=2,
  ntt=3,
  gamma_ln=1.0,
  temp0=150.0,
  nmropt=1,
  ntb=1,
  ntp=0,
  cut=8.0,
  ntr=1,
  restraint_wt=125.0,
  restraintmask='!@H= & !:WAT & !:Na+',
  ntpr=5000,
  ntwx=5000,
  ntwe=5000,
  ntwr=50000,
  ioutfm=1,
/
&wt
  TYPE='TEMP0', ISTEP1=0, ISTEP2=50000, VALUE1=100.0, VALUE2=150.0,
```

```
/
&wt TYPE='END' /
EOF
```

heat3.in (100 -> 150 K)

```
cat > $WORK/runs/pftrps_ain_md/heat4.in << 'EOF'
PfTrpS(Ain) heat4; SI Heating 150->200 K, 85 kcal/mol/A^2
&cntrl
  imin=0,
  irest=1,
  ntx=5,
  nstlim=50000,
  dt=0.001,
  ntc=2,
  ntf=2,
  ntt=3,
  gamma_ln=1.0,
  temp0=200.0,
  nmropt=1,
  ntb=1,
  ntp=0,
  cut=8.0,
  ntr=1,
  restraint_wt=85.0,
  restraintmask='!@H= & !:WAT & !:Na+',
  ntpr=5000,
  ntwx=5000,
  ntwe=5000,
  ntwr=50000,
  ioutfm=1,
/
&wt
  TYPE='TEMP0', ISTEP1=0, ISTEP2=50000, VALUE1=150.0, VALUE2=200.0,
/
&wt TYPE='END' /
EOF
```

heat4.in (150 -> 200 K)

```
cat > $WORK/runs/pftrps_ain_md/heat5.in << 'EOF'
PfTrpS(Ain) heat5; SI Heating 200->250 K, 45 kcal/mol/A^2
&cntrl
  imin=0,
  irest=1,
```

```
  ntx=5,
  nstlim=50000,
  dt=0.001,
  ntc=2,
  ntf=2,
  ntt=3,
  gamma_ln=1.0,
  temp0=250.0,
  nmropt=1,
  ntb=1,
  ntp=0,
  cut=8.0,
  ntr=1,
  restraint_wt=45.0,
  restraintmask='!@H= & !:WAT & !:Na+',
  ntpr=5000,
  ntwx=5000,
  ntwe=5000,
  ntwr=50000,
  ioutfm=1,
/
&wt
  TYPE='TEMP0', ISTEP1=0, ISTEP2=50000, VALUE1=200.0, VALUE2=250.0,
/
&wt TYPE='END' /
EOF
```

heat5.in (200 -> 250 K)

```
cat > $WORK/runs/pftrps_ain_md/heat6.in << 'EOF'
PfTrpS(Ain) heat6; SI Heating 250->300 K, 10 kcal/mol/A^2
&cntrl
  imin=0,
  irest=1,
  ntx=5,
  nstlim=50000,
  dt=0.001,
  ntc=2,
  ntf=2,
  ntt=3,
  gamma_ln=1.0,
  temp0=300.0,
  nmropt=1,
  ntb=1,
  ntp=0,
  cut=8.0,
```

```
  ntr=1,
  restraint_wt=10.0,
  restraintmask='!@H= & !:WAT & !:Na+',
  ntpr=5000,
  ntwx=5000,
  ntwe=5000,
  ntwr=50000,
  ioutfm=1,
/
&wt
  TYPE='TEMP0', ISTEP1=0, ISTEP2=50000, VALUE1=250.0, VALUE2=300.0,
/
&wt TYPE='END' /
EOF
```

heat6.in (250 -> 300 K)

heat7.in (300 -> 350 K)

> **Note**: SI lists 6 restraint values for 7 steps. heat6 and heat7 both use 10.0 kcal/mol/A^2. `[operational default]`

```
cat > $WORK/runs/pftrps_ain_md/heat7.in << 'EOF'
PfTrpS(Ain) heat7; SI Heating 300->350 K, 10 kcal/mol/A^2
&cntrl
  imin=0,
  irest=1,
  ntx=5,
  nstlim=50000,
  dt=0.001,
  ntc=2,
  ntf=2,
  ntt=3,
  gamma_ln=1.0,
  temp0=350.0,
  nmropt=1,
  ntb=1,
  ntp=0,
  cut=8.0,
  ntr=1,
  restraint_wt=10.0,
  restraintmask='!@H= & !:WAT & !:Na+',
  ntpr=5000,
  ntwx=5000,
  ntwe=5000,
  ntwr=50000,
  ioutfm=1,
/
```

```
&wt
  TYPE='TEMP0', ISTEP1=0, ISTEP2=50000, VALUE1=300.0, VALUE2=350.0,
/
&wt TYPE='END' /
EOF
```

Heating Step Summary

Step	Temperature	Restraint (kcal/mol/A^2)	Source
heat1	0 -> 50 K	210.0	[SI p.S2]
heat2	50 -> 100 K	165.0	[SI p.S2]
heat3	100 -> 150 K	125.0	[SI p.S2]
heat4	150 -> 200 K	85.0	[SI p.S2]
heat5	200 -> 250 K	45.0	[SI p.S2]
heat6	250 -> 300 K	10.0	[SI p.S2]
heat7	300 -> 350 K	10.0	[operational default]

Step 4.3: Create equilibration input file

What: 2 ns NPT equilibration at 350 K and 1 atm with no positional restraints.

Why NPT after NVT heating? During heating (NVT), the box volume was fixed, so density may not be realistic. NPT lets the box expand or shrink to reach the correct density (~1.0 g/cm^3 for a water-solvated protein at 1 atm).

Why: After heating, the system is at the target temperature but the box volume/density may not be at equilibrium. NPT equilibration allows the box to adjust its size to achieve the correct density under constant pressure. [SI p.S3]

```
cat > $WORK/runs/pftrps_ain_md/equil.in << 'EOF'
PfTrpS(Ain) equil; SI 2 ns NPT, 350 K, 1 atm
&cntrl
  imin=0,
  irest=1,
  ntx=5,
  nstlim=1000000,
  dt=0.002,
  ntc=2,
  ntf=2,
  ntt=3,
  gamma_ln=1.0,
  temp0=350.0,
  ntb=2,
  ntp=1,
  barostat=1,
  pres0=1.0,
  taup=1.0,
```

```
  cut=8.0,
  ntr=0,
  ntpr=5000,
  ntwx=5000,
  ntwe=5000,
  ntwr=50000,
  ioutfm=1,
/
EOF
```

Parameter	Value	Meaning	Source
`nstlim=1000000`	1000000	1,000,000 steps x 0.002 ps = 2 ns	`[SI p.S3]`
`dt=0.002`	0.002	2 fs timestep (standard with SHAKE)	`[SI p.S3]`
`ntb=2`	2	Constant pressure periodic boundaries (NPT)	`[SI p.S3]`
`ntp=1`	1	Isotropic pressure coupling	`[SI p.S3]`
`barostat=1`	1	Berendsen barostat	`[operational default]`
`pres0=1.0`	1.0	Target pressure: 1 atm	`[operational default]`
`taup=1.0`	1.0	Pressure relaxation time: 1 ps	`[operational default]`
`ntr=0`	0	No positional restraints	`[SI p.S3]`

`ntb=1` vs `ntb=2`: `ntb=1` = constant volume (NVT, used for heating and production). `ntb=2` = constant pressure (NPT, used for equilibration to find the correct density). Different simulation phases need different ensembles.

What is a barostat? A barostat controls pressure by adjusting the simulation box volume. When pressure is too high, the box expands; when too low, it shrinks. Used during NPT equilibration.

Step 4.4: Create the prep pipeline Slurm script

What: A single Slurm job that runs min1 -> min2 -> heat1-7 -> equil sequentially on a GPU node.

Why: Each step depends on the restart file from the previous step. Running them as one job avoids queue wait times between steps.

```
cat > $WORK/runs/pftrps_ain_md/run_md_pipeline.sh << 'EOF'
#!/bin/bash
#SBATCH --job-name=pftrps_ain_prep
#SBATCH --partition=gpu
```

```
#SBATCH --qos=gpu_access
#SBATCH --gres=gpu:1
#SBATCH --nodes=1
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=4
#SBATCH --mem=16G
#SBATCH --time=24:00:00
#SBATCH --output=$WORK/logs/pftrps_ain_prep_%j.out

# ===== Environment =====
module load amber/24p3

RUNDIR="$WORK/runs/pftrps_ain_md"
PARM="$WORK/systems/pftrps_ain/pftrps_ain.parm7"
INPCRD="$WORK/systems/pftrps_ain/pftrps_ain.inpcrd"

cd ${RUNDIR}

echo "========== PREP PIPELINE START: $(date) =========="

# ===== Minimization Stage 1: Restrained =====
echo "--- min1: restrained minimization ---"
pmemd.cuda -O \
  -i min1.in \
  -o min1.out \
  -p ${PARM} \
  -c ${INPCRD} \
  -r min1.rst7 \
  -ref ${INPCRD}
echo "min1 done: $(date)"

# ===== Minimization Stage 2: Unrestrained =====
echo "--- min2: unrestrained minimization ---"
pmemd.cuda -O \
  -i min2.in \
  -o min2.out \
  -p ${PARM} \
  -c min1.rst7 \
  -r min2.rst7
echo "min2 done: $(date)"

# ===== Heating Steps 1-7 =====
PREV_RST="min2.rst7"
for i in 1 2 3 4 5 6 7; do
  echo "--- heat${i} ---"
  pmemd.cuda -O \
    -i heat${i}.in \
```

```
    -o heat${i}.out \
    -p ${PARM} \
    -c ${PREV_RST} \
    -r heat${i}.rst7 \
    -x heat${i}.nc \
    -ref ${PREV_RST}
  PREV_RST="heat${i}.rst7"
  echo "heat${i} done: $(date)"
done

# ===== Equilibration (2 ns NPT) =====
echo "--- equil: 2 ns NPT equilibration ---"
pmemd.cuda -O \
  -i equil.in \
  -o equil.out \
  -p ${PARM} \
  -c heat7.rst7 \
  -r equil.rst7 \
  -x equil.nc
echo "equil done: $(date)"

echo "========== PREP PIPELINE COMPLETE: $(date) =========="
EOF
```

Step 4.5: Submit the prep pipeline

```
cd $WORK/runs/pftrps_ain_md
sbatch run_md_pipeline.sh
```

> **If this fails:** If the job fails immediately, check the Slurm output: `cat $WORK/logs/pftrps_ain_prep_*.o`
> Common issues: (1) wrong partition name – run `sinfo` to see available partitions; (2) insufficient memory or walltime; (3) module not loaded – ensure `module load amber/24p3` is in the Slurm script; (4) GPU not available – check `--gres=gpu:1` and `--qos=gpu_access` match your HPC's configuration.

Verification (after job completes):

```
# Check all output files exist:
ls -lh $WORK/runs/pftrps_ain_md/{min1,min2}.out
ls -lh $WORK/runs/pftrps_ain_md/heat{1,2,3,4,5,6,7}.out
ls -lh $WORK/runs/pftrps_ain_md/equil.out

# Check restart files:
ls -lh $WORK/runs/pftrps_ain_md/\
  {min1,min2,heat1,heat2,heat3,heat4,heat5,heat6,heat7,equil}.rst7

# Check minimization converged (energy should decrease):
grep "NSTEP       ENERGY" $WORK/runs/pftrps_ain_md/min1.out | tail -3
```

```
# Check final temperature after heat7 is ~350 K:
grep "TEMP(K)" $WORK/runs/pftrps_ain_md/heat7.out | tail -1
# Expected: ~350 K (within fluctuations)

# Check equilibration density is ~1.0 g/cm^3:
grep "Density" $WORK/runs/pftrps_ain_md/equil.out | tail -5
# Expected: ~1.0 g/cm^3

# Check for errors:
grep -l "ERROR\|FATAL\|NaN" $WORK/runs/pftrps_ain_md/*.out
# Expected: (no output = good)
```

Phase 5: Classical MD – Production 500 ns (~3 days)

Goal: Run 500 ns of unbiased NVT molecular dynamics at 350 K to sample the Ain conformational ensemble.

Why: The conventional MD trajectory serves two purposes: (1) it provides the equilibrated starting structure for MetaDynamics, and (2) it enables comparison of the sampled conformational space with and without enhanced sampling bias. [SI p.S3]

Step 5.1: Create production input file

```
cat > $WORK/runs/pftrps_ain_md/prod.in << 'EOF'
PfTrpS(Ain) prod; SI 500 ns NVT, 350 K, 8 A cutoff
&cntrl
  imin=0,
  irest=1,
  ntx=5,
  nstlim=250000000,
  dt=0.002,
  ntc=2,
  ntf=2,
  ntt=3,
  gamma_ln=1.0,
  temp0=350.0,
  ntb=1,
  ntp=0,
  cut=8.0,
  ntr=0,
  ntpr=5000,
  ntwx=5000,
  ntwe=500,
  ntwr=500000,
  ioutfm=1,
```

```
/
EOF
```

Parameter	Value	Meaning	Source
`nstlim=250000000`	250,000,000	250M steps x 0.002 ps = 500 ns	`[SI p.S3]`
`ntb=1`	1	NVT (constant volume) for production	`[SI p.S3]`
`ntwe=500`	500	Write energy every 500 steps (= 1 ps, fine-grained energy monitoring)	`[operational default]`
`ntwr=500000`	500000	Write restart every 500K steps (= 1 ns, for crash recovery)	`[operational default]`
`ntwx=5000`	5000	Write trajectory frame every 5000 steps (= 10 ps)	`[operational default]`

Trajectory size estimate: 500 ns / 10 ps = 50,000 frames. At ~39,000 atoms, each NetCDF frame is ~0.5 MB. Total: ~25 GB.

Step 5.2: Create production Slurm script

```
cat > $WORK/runs/pftrps_ain_md/submit_production.sh << 'EOF'
#!/bin/bash
#SBATCH --job-name=pftrps_ain_prod
#SBATCH --partition=gpu
#SBATCH --qos=gpu_access
#SBATCH --gres=gpu:1
#SBATCH --nodes=1
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=4
#SBATCH --mem=16G
#SBATCH --time=72:00:00
#SBATCH --output=$WORK/logs/pftrps_ain_prod_%j.out

module load amber/24p3

RUNDIR="$WORK/runs/pftrps_ain_md"
PARM="$WORK/systems/pftrps_ain/pftrps_ain.parm7"

cd ${RUNDIR}

echo "========== PRODUCTION START: $(date) ==========="
```

```
pmemd.cuda -O \
  -i prod.in \
  -o prod.out \
  -p ${PARM} \
  -c equil.rst7 \
  -r prod.rst7 \
  -x prod.nc

echo "========== PRODUCTION COMPLETE: $(date) =========="
EOF
```

Step 5.3: Submit production

```
sbatch $WORK/runs/pftrps_ain_md/submit_production.sh
```

> **If this fails:** If the job fails immediately, check `cat $WORK/logs/pftrps_ain_prod_*.out`. Common issues: (1) missing `equil.rst7` – the prep pipeline must complete first; (2) "CUDA out of memory" – request a GPU with more VRAM (e.g., `--gres=gpu:a100:1`); (3) walltime exceeded – request more time or split into multiple shorter runs using restart files.

Verification (during and after run):

```
# Monitor job status:
squeue -u $USER

# Check progress (number of frames written so far):
# The trajectory file grows over time
ls -lh $WORK/runs/pftrps_ain_md/prod.nc

# After completion, check for normal termination:
tail -20 $WORK/runs/pftrps_ain_md/prod.out | grep "Total wall time"
# Should show total wall time

# Verify trajectory frame count:
module load amber/24p3
cpptraj -p $WORK/systems/pftrps_ain/pftrps_ain.parm7 \
  -y $WORK/runs/pftrps_ain_md/prod.nc \
  -tl
# Expected: 50,000 frames

# Quick RMSD check (should plateau if equilibrated):
cpptraj -p $WORK/systems/pftrps_ain/pftrps_ain.parm7 << CPPTRAJ_EOF
trajin $WORK/runs/pftrps_ain_md/prod.nc
rms first @CA out $WORK/analysis/prod_rmsd.dat
go
CPPTRAJ_EOF
head -20 $WORK/analysis/prod_rmsd.dat
```

Phase 6: AMBER to GROMACS Conversion [DRAFT – not yet verified] (~30 min)

Goal: Convert the equilibrated AMBER system (parm7 + restart) to GROMACS format (gro + top) for MetaDynamics with PLUMED.

Why: The original JACS 2019 study used GROMACS 5.1.2 + PLUMED 2 for MetaDynamics. We replicate this choice strictly: conventional MD in AMBER (equivalent ff14SB), MetaDynamics in GROMACS + PLUMED. [SI p.S3]

Step 6.1: Convert topology with ParmEd

What: ParmEd can read AMBER parm7/inpcrd and write GROMACS top/gro files.

```
cat > $WORK/scripts/convert_amber_to_gromacs.py << 'EOF'
#!/usr/bin/env python3
"""
Convert AMBER topology/coordinates to GROMACS format using ParmEd.
[DRAFT -- not yet verified]

Input:  pftrps_ain.parm7 + equil.rst7 (or prod.rst7)
Output: pftrps_ain.top + pftrps_ain.gro
"""
import parmed as pmd

# Load AMBER files
parm = pmd.load_file(
    "$WORK/systems/pftrps_ain/pftrps_ain.parm7",
    "$WORK/runs/pftrps_ain_md/equil.rst7"
)

# Save as GROMACS format
parm.save("$WORK/systems/pftrps_ain/pftrps_ain.top", overwrite=True)
parm.save("$WORK/systems/pftrps_ain/pftrps_ain.gro", overwrite=True)

print("Conversion complete.")
print(f"Atoms: {len(parm.atoms)}")
print(f"Residues: {len(parm.residues)}")
EOF

module load anaconda/2024.02 && conda activate trpb-md
python3 $WORK/scripts/convert_amber_to_gromacs.py
```

Verification:

```
ls -lh $WORK/systems/pftrps_ain/pftrps_ain.{top,gro}
# Both files should exist
```

```
# Check atom count in .gro matches AMBER:
head -2 $WORK/systems/pftrps_ain/pftrps_ain.gro
# Second line should show total atom count (e.g., 39268)

# Check box vectors in .gro (last line):
tail -1 $WORK/systems/pftrps_ain/pftrps_ain.gro
# Should show box dimensions in nm (divide AMBER Angstroms by 10)
```

Step 6.2: Generate GROMACS .tpr file

What: Create a GROMACS binary run input file (.tpr) that combines topology, coordinates, and simulation parameters.

```
cat > $WORK/runs/pftrps_ain_metad/em.mdp << 'EOF'
; Energy minimization MDP for GROMACS (post-conversion sanity check)
; [DRAFT -- not yet verified]
integrator = steep
emtol      = 1000.0
emstep     = 0.01
nsteps     = 5000
nstlist    = 1
cutoff-scheme = Verlet
coulombtype = PME              ; see PME note below
rcoulomb    = 0.8
rvdw        = 0.8
pbc         = xyz
EOF
```

What is PME? PME (Particle Mesh Ewald) is a fast algorithm for calculating electrostatic (charge-charge) interactions between all atoms, including those far apart. Without PME, these calculations would be impossibly slow for large systems.

```
module load anaconda/2024.02 && conda activate trpb-md

gmx grompp \
  -f $WORK/runs/pftrps_ain_metad/em.mdp \
  -c $WORK/systems/pftrps_ain/pftrps_ain.gro \
  -p $WORK/systems/pftrps_ain/pftrps_ain.top \
  -o $WORK/runs/pftrps_ain_metad/em.tpr \
  -maxwarn 5
```

Verification:

```
ls -lh $WORK/runs/pftrps_ain_metad/em.tpr
# Should exist, ~2-5 MB

# Quick energy minimization to test topology:
gmx mdrun -v -deffnm $WORK/runs/pftrps_ain_metad/em -ntomp 4
# Should complete without errors
```

FP-004 warning: After conversion, all atom indices change. PLUMED input files must use the GROMACS atom numbering, not the original PDB or AMBER numbering.

Phase 7: Path CV Construction [DRAFT – not yet verified] (~10 min)

What is a path collective variable (path CV)? A path CV measures how similar the current protein structure is to a series of reference structures arranged along a pathway. s(R) tells you "where along the path are you?" (1 = open, 15 = closed). z(R) tells you "how far off the path are you?"

Goal: Build the reference path for the PATHMSD collective variable, consisting of 15 intermediate frames between the Open (1WDW) and Closed (3CEP) conformations of the COMM domain.

What the path CV measures: Two quantities: - **s(R)**: Progress along the O -> C conformational path (ranges from 1 to 15) - **z(R)**: Distance from the nearest reference frame (measures how far the current structure deviates from the reference path)

Source: `[SI p.S3]` – 15 frames, Calpha atoms of residues 97-184 + 282-305

Step 7.1: Align open and closed structures

What: Superimpose 1WDW (open) and 3CEP (closed) on their COMM domain Calpha atoms, then linearly interpolate 15 frames.

Why: The path CV requires a series of reference structures that smoothly connect the two endpoints. Linear interpolation in Cartesian space is the simplest approach. `[SI p.S3]`

```
cat > $WORK/scripts/generate_path_cv.py << 'EOF'
#!/usr/bin/env python3
"""
Generate 15-frame reference path from 1WDW (open) to 3CEP (closed).
[DRAFT -- not yet verified]

Atoms used: Calpha of residues 97-184 and 282-305 [SI p.S3]
Interpolation: Linear in Cartesian coordinates [SI p.S3]

Output: path.pdb (multi-model PDB for PLUMED PATHMSD)
"""
import numpy as np
import warnings

# [DRAFT] This script needs:
# 1. MDAnalysis or BioPython to read PDBs
# 2. Sequence alignment to map 1WDW residues to 3CEP residues
#    (3CEP is S. typhimurium TrpS, different species)
# 3. Calpha extraction for residues 97-184 and 282-305
```

```
# 4. Rigid-body superposition (Kabsch algorithm)
# 5. Linear interpolation of 15 frames
# 6. MSD calculation between adjacent frames
# 7. Lambda calculation: lambda = 2.3 / mean_MSD [SI p.S3]

print("[DRAFT] Path CV generation script -- not yet implemented")
print("Expected output: path.pdb with 15 MODELs")
print("Expected lambda: ~0.034 A^-2 (our calculation) vs ~0.029 (SI)")
EOF
```

Step 7.2: Lambda calculation

Where does 2.3 come from? The value 2.3 is the standard scaling factor used in PLUMED's PATHMSD implementation. It comes from the original path CV paper (Branduardi et al., 2007). lambda = 2.3 / MSD ensures that the path CV responds smoothly to structural changes between adjacent frames.

Key formula: `lambda = 2.3 / <MSD>` where `<MSD>` is the mean squared displacement between adjacent path frames, averaged over all 14 intervals. `[SI p.S3]`

Value	SI Reported	Our Calculation	Source
Mean MSD	80 A^2	67.826 A^2	`[SI p.S3]`
Lambda	0.029 A^-2	0.033910 A^-2	Derived from MSD

Discrepancy note: The 15% difference in MSD likely arises from different superposition methods. The SI does not specify which atoms were used for alignment before computing MSD. Our current value (0.033910) uses total MSD across all selected Calpha atoms.

FP-015 guard: The `calculate_msd()` function must return MSD (units: A^2), NOT RMSD (units: A). A previous bug applied `np.sqrt()` to the MSD, producing RMSD, which gave lambda = 3.798 instead of 0.034.

Phase 8: Well-Tempered MetaDynamics (~2–3 days)

What is MetaDynamics? MetaDynamics is a simulation technique that adds small energy "bumps" (Gaussian hills) to already-visited states, pushing the system to explore new conformations it would otherwise take millions of years to reach. Over time, the accumulated bumps fill up energy basins, and the negative of the total bias gives you the free energy landscape.

What is well-tempered MetaDynamics? In standard MetaDynamics, hills keep growing forever. In well-tempered MetaDynamics, hill height decreases over time as the bias grows, preventing over-filling and allowing the simulation to converge to the correct free energy surface.

Goal: Run well-tempered MetaDynamics with path CVs using GROMACS + PLUMED to sample the full Open-to-Closed conformational landscape of the COMM domain.

Protocol: Single-walker exploration first, then 10-walker production. `[SI pp.S3–S4]`

CRITICAL: Lessons learned from debugging (2026-04-04)

Three compatibility issues were discovered when running PLUMED path CVs with GROMACS 2026. These are documented below and reflected in all code blocks in this section.

1. **FUNCPATHMSD replaces PATHMSD**: The classic `PATHMSD` action reads a multi-MODEL PDB reference file internally. However, GROMACS 2026's `mdrun` interface does not correctly parse PDB files with non-sequential atom serial numbers (e.g., Calpha-only selections). The workaround is `FUNCPATHMSD`: define 15 individual `RMSD` actions (one per reference frame), then pass them to `FUNCPATHMSD` which computes the mathematically identical path CV formula.
2. **LAMBDA unit conversion (Angstrom to nm)**: LAMBDA is computed as `1 / (N-1) / <MSD>` where MSD is in the native length unit squared. The original paper (AMBER/Angstrom) uses LAMBDA in A^-2. GROMACS uses nanometers internally, so **LAMBDA must be multiplied by 100** (since 1 nm^2 = 100 A^2). Our calculated value: 0.033910 A^-2 –> **3.3910 nm^-2**.
3. **No backslash line continuation**: When PLUMED input is passed via `gmx mdrun -plumed`, backslash (`\`) continuation across lines is not reliably parsed. All parameters for a single PLUMED action must appear on **one line**. The code blocks below use single-line format.
4. **Source-compiled PLUMED required**: The conda-forge PLUMED package ships an incomplete `libplumedKernel.so` that causes crashes at runtime. PLUMED must be compiled from source with `./configure --enable-modules=all` and the resulting library path exported via `PLUMED_KERNEL`.

Step 8.1: Create GROMACS MDP for MetaDynamics

```
# Directory names are your choice -- adjust paths to match your project layout.
mkdir -p $WORK/runs/pftrps_ain_metad

cat > $WORK/runs/pftrps_ain_metad/metad.mdp << 'EOF'
; GROMACS MDP for well-tempered MetaDynamics
;
; Reference: JACS 2019 SI p.S3
; Engine: GROMACS 5.1.2 (original) -> GROMACS 2026.0 (ours)

integrator   = md              ; Leap-frog integrator
dt           = 0.002           ; 2 fs timestep [SI p.S3]
nsteps       = 25000000        ; 25M steps = 50 ns per walker [SI p.S4]

; Output control
```

```
nstxout-compressed = 5000      ; Write compressed trajectory every 10 ps
nstlog       = 5000            ; Write log every 10 ps
nstenergy    = 5000            ; Write energy every 10 ps

; Neighbor searching
nstlist      = 10              ; Update neighbor list every 10 steps
cutoff-scheme = Verlet          ; Verlet cutoff scheme (GROMACS default)

; Electrostatics
coulombtype  = PME             ; Particle Mesh Ewald [SI p.S3]
rcoulomb     = 0.8             ; 8 A cutoff in nm [SI p.S3]

; Van der Waals
rvdw         = 0.8             ; 8 A cutoff in nm [SI p.S3]

; Temperature coupling
tcoupl       = v-rescale       ; Velocity rescaling thermostat[^4]
tc-grps      = System          ; Single coupling group
tau_t        = 0.1             ; Coupling time constant (ps)
ref_t        = 350             ; 350 K [SI p.S3]

; Pressure coupling
pcoupl       = no              ; NVT (no pressure coupling) [SI p.S3]

; Bond constraints
constraints  = h-bonds         ; Constrain H-bonds (SHAKE) [SI p.S3]
constraint_algorithm = LINCS   ; LINCS algorithm (GROMACS default)

; Continuation from AMBER equilibration
continuation = yes             ; Do not generate velocities
gen_vel      = no              ; Read from .gro / .tpr
EOF
```

Step 8.2: Create PLUMED input for single-walker MetaDynamics

MetaDynamics parameters explained:

Parameter	Value	Meaning	Source
`HEIGHT=0.628`	0.628 kJ/mol	Hill height. SI reports 0.15 kcal/mol; 0.15 x 4.184 = 0.628 kJ/mol (PLUMED uses kJ/mol).	`[SI p.S3]`

Parameter	Value	Meaning	Source
`PACE=1000`	every 1000 steps	Deposit a new hill every 1000 MD steps. At dt=0.002 ps, this is every 2 ps.	`[SI p.S3]`
`BIASFACTOR=10`	10	Well-tempered MetaD bias factor. The effective temperature of the CV is T_eff = T * (1 + 1/gamma) = 350 * 1.1 = 385 K. Higher = more exploration, less precise free energies.	`[SI p.S3]`
`TEMP=350`	350 K	System temperature (must match MD thermostat).	`[SI p.S3]`
`SIGMA=0.2,0.1`	0.2 (s), 0.1 (z)	Gaussian widths for the s and z CVs respectively.	`[SI p.S3]`
`LAMBDA=3.3910`	3.3910 nm^-2	FUNCPATHMSD smoothing parameter. Controls how sharply each reference frame "attracts" structures. **Converted from 0.033910 A^-2 x 100 = 3.3910 nm^-2 for GROMACS.**	Calculated from our MSD; SI reports 0.029 A^-2

LAMBDA unit conversion (CRITICAL): The smoothing parameter LAMBDA has units of inverse-length-squared. Our calculated value is 0.033910 A^-2. GROMACS uses nanometers internally, so all distances fed to PLUMED are in nm. Since 1 nm = 10 A, we have 1 nm^-2 = 0.01 A^-2, therefore **multiply by 100**: 0.033910 A^-2 x 100 = **3.3910 nm^-2**. Using the unconverted value (0.033910) would make all frames appear nearly equidistant and the path CV would not discriminate conformations.

Unit conversion: The paper reports hill height as 0.15 kcal/mol, but PLUMED uses kJ/mol internally. Conversion: 0.15 kcal/mol x 4.184 = 0.628 kJ/mol. Always use kJ/mol in PLUMED input files.

How PACE relates to time: PACE=1000 means deposit a Gaussian hill every 1000 MD steps. With dt=0.002 ps, that's 1000 x 0.002 = 2.0 ps between hills. This matches

the SI specification.

What is BIASFACTOR? It controls how quickly hill heights decrease in well-tempered MetaDynamics. BIASFACTOR=10 means the effective temperature for the CV is T x (1 + 1/gamma) = 350 x 1.1 = 385 K. Higher values explore more aggressively but converge slower.

Why FUNCPATHMSD instead of PATHMSD? The classic `PATHMSD` action reads a multi-MODEL PDB file internally. When GROMACS 2026 `mdrun` passes coordinates to PLUMED, atom serial numbers may be non-sequential (e.g., Calpha-only selections skip most atoms). The `PATHMSD` PDB parser does not handle this correctly, causing silent misalignment or crashes. The workaround: define 15 individual `RMSD` actions (one per reference frame PDB), then feed them into `FUNCPATHMSD`. This computes the **mathematically identical** s(R) and z(R) using the same exponential formula, but avoids the PDB parsing issue entirely.

No backslash continuation: When PLUMED input is read via the `gmx mdrun -plumed` interface, backslash line continuation (\) is not reliably parsed. All keywords for a single PLUMED action must be on **one line**. The code below follows this convention.

```
# Prepare per-frame reference PDB directory
# Each frame_XX.pdb contains Calpha atoms of COMM domain residues 97-184, 282-305
# with coordinates in NANOMETERS (GROMACS convention)
mkdir -p $WORK/runs/pftrps_ain_metad/frames
# (copy your 15 reference PDBs here: frame_01.pdb ... frame_15.pdb)

cat > $WORK/runs/pftrps_ain_metad/plumed_trpb_metad.dat << 'EOF'
# PLUMED 2.9 single-walker WT-MetaD for PfTrpS(Ain)
# Verified 2026-04-04
#
# Reference: JACS 2019 SI pp.S3-S4
# CVs: path collective variables s(R) and z(R) via FUNCPATHMSD
#   s(R) = progress along O->C path (ranges 1-15)
#   z(R) = deviation from reference path
#
# FP-004 guard: atom indices in frame PDBs must match GROMACS numbering,
# not PDB or AMBER numbering.
#
# NOTE: All PLUMED actions on single lines (no backslash continuation)
# because gmx mdrun -plumed does not reliably parse line continuations.

# 15 individual RMSD references (one per frame along O->C path)
r1: RMSD REFERENCE=frames/frame_01.pdb TYPE=OPTIMAL
r2: RMSD REFERENCE=frames/frame_02.pdb TYPE=OPTIMAL
r3: RMSD REFERENCE=frames/frame_03.pdb TYPE=OPTIMAL
r4: RMSD REFERENCE=frames/frame_04.pdb TYPE=OPTIMAL
r5: RMSD REFERENCE=frames/frame_05.pdb TYPE=OPTIMAL
r6: RMSD REFERENCE=frames/frame_06.pdb TYPE=OPTIMAL
r7: RMSD REFERENCE=frames/frame_07.pdb TYPE=OPTIMAL
```

```
r8: RMSD REFERENCE=frames/frame_08.pdb TYPE=OPTIMAL
r9: RMSD REFERENCE=frames/frame_09.pdb TYPE=OPTIMAL
r10: RMSD REFERENCE=frames/frame_10.pdb TYPE=OPTIMAL
r11: RMSD REFERENCE=frames/frame_11.pdb TYPE=OPTIMAL
r12: RMSD REFERENCE=frames/frame_12.pdb TYPE=OPTIMAL
r13: RMSD REFERENCE=frames/frame_13.pdb TYPE=OPTIMAL
r14: RMSD REFERENCE=frames/frame_14.pdb TYPE=OPTIMAL
r15: RMSD REFERENCE=frames/frame_15.pdb TYPE=OPTIMAL

# Path CV via FUNCPATHMSD -- mathematically identical to PATHMSD
# LAMBDA=3.3910 nm^-2 (= 0.033910 A^-2 x 100, unit conversion for GROMACS)
path: FUNCPATHMSD ARG=r1,r2,r3,r4,r5,r6,r7,r8,r9,r10,r11,r12,r13,r14,r15 LAMBDA=3.3910

# Well-tempered MetaDynamics biasing s(R) and z(R) simultaneously
metad: METAD ARG=path.s,path.z SIGMA=0.2,0.1 HEIGHT=0.628 PACE=1000 BIASFACTOR=10 TEMP

# Print CVs and bias for analysis
PRINT ARG=path.s,path.z,metad.bias FILE=COLVAR STRIDE=500
EOF
```

Step 8.3: Create Slurm script for MetaDynamics

> **Source-compiled PLUMED**: The conda-forge PLUMED package ships an incomplete `libplumedKernel.so` that causes runtime crashes when GROMACS tries to load it. You must compile PLUMED from source (`./configure --enable-modules=all && make -j8 && make install`) and export the kernel path. The Slurm script below assumes PLUMED was installed to `$WORK/software/plumed2`.

```
cat > $WORK/runs/pftrps_ain_metad/submit_metad.slurm << 'EOF'
#!/bin/bash
#SBATCH --job-name=pftrps_metad
#SBATCH --partition=gpu
#SBATCH --qos=gpu_access
#SBATCH --gres=gpu:1
#SBATCH --nodes=1
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=4
#SBATCH --mem=16G
#SBATCH --time=72:00:00
#SBATCH --output=$WORK/logs/pftrps_metad_%j.out

# Load environment
module load anaconda/2024.02
eval "$(conda shell.bash hook)"
conda activate trpb-md

# Use source-compiled PLUMED (conda-forge libplumedKernel.so is incomplete)
```

```
export PLUMED_KERNEL=$WORK/software/plumed2/lib/libplumedKernel.so

# FP-006 guard: set OMP_NUM_THREADS to match cpus-per-task
export OMP_NUM_THREADS=${SLURM_CPUS_PER_TASK}

RUNDIR="$WORK/runs/pftrps_ain_metad"
cd ${RUNDIR}

echo "========== METAD START: $(date) ==========="

# Generate .tpr
gmx grompp \
  -f metad.mdp \
  -c $WORK/systems/pftrps_ain/pftrps_ain.gro \
  -p $WORK/systems/pftrps_ain/pftrps_ain.top \
  -o metad.tpr \
  -maxwarn 5

# Run with PLUMED
gmx mdrun -v \
  -deffnm metad \
  -plumed plumed_trpb_metad.dat \
  -ntomp ${SLURM_CPUS_PER_TASK}

echo "========== METAD COMPLETE: $(date) ==========="
EOF
```

Step 8.4: Multi-walker protocol (10 walkers)

> **What**: Run 10 independent MetaDynamics walkers that share their HILLS files. Each walker deposits hills in a shared directory that all walkers read from. This accelerates convergence because each walker benefits from the bias accumulated by all others. [SI p.S4]

```
cat > $WORK/runs/pftrps_ain_metad/plumed_trpb_metad_multiwalker.dat << 'EOF'
# PLUMED 2.9 multi-walker WT-MetaD for PfTrpS(Ain)
# Verified 2026-04-04
#
# 10 walkers, 50-100 ns each = 500-1000 ns total [SI p.S4]
# Walkers share HILLS via filesystem
#
# NOTE: All PLUMED actions on single lines (no backslash continuation)

r1: RMSD REFERENCE=frames/frame_01.pdb TYPE=OPTIMAL
r2: RMSD REFERENCE=frames/frame_02.pdb TYPE=OPTIMAL
r3: RMSD REFERENCE=frames/frame_03.pdb TYPE=OPTIMAL
r4: RMSD REFERENCE=frames/frame_04.pdb TYPE=OPTIMAL
```

```
r5: RMSD REFERENCE=frames/frame_05.pdb TYPE=OPTIMAL
r6: RMSD REFERENCE=frames/frame_06.pdb TYPE=OPTIMAL
r7: RMSD REFERENCE=frames/frame_07.pdb TYPE=OPTIMAL
r8: RMSD REFERENCE=frames/frame_08.pdb TYPE=OPTIMAL
r9: RMSD REFERENCE=frames/frame_09.pdb TYPE=OPTIMAL
r10: RMSD REFERENCE=frames/frame_10.pdb TYPE=OPTIMAL
r11: RMSD REFERENCE=frames/frame_11.pdb TYPE=OPTIMAL
r12: RMSD REFERENCE=frames/frame_12.pdb TYPE=OPTIMAL
r13: RMSD REFERENCE=frames/frame_13.pdb TYPE=OPTIMAL
r14: RMSD REFERENCE=frames/frame_14.pdb TYPE=OPTIMAL
r15: RMSD REFERENCE=frames/frame_15.pdb TYPE=OPTIMAL

path: FUNCPATHMSD ARG=r1,r2,r3,r4,r5,r6,r7,r8,r9,r10,r11,r12,r13,r14,r15 LAMBDA=3.3910

metad: METAD ARG=path.s,path.z SIGMA=0.2,0.1 HEIGHT=0.628 PACE=1000 BIASFACTOR=10 TEMP

PRINT ARG=path.s,path.z,metad.bias FILE=COLVAR STRIDE=500
EOF
```

PLUMED Multi-Walker Parameter	Value	Meaning	Source
`WALKERS_N=10`	10	Total number of walkers	`[SI p.S4]`
`WALKERS_ID=__WALKER_ID__`	0-9	This walker's ID (substituted by launch script)	–
`WALKERS_DIR=../shared_bias`	–	Directory where all walkers read/write HILLS	–
`WALKERS_RSTRIDE=1000`	1000	Check for new hills from other walkers every 1000 steps	`[operational default]`

> **Walker starting structures**: Extract 10 snapshots from the initial single-walker Meta-Dynamics run. These should span different regions of the s(R) CV to maximize exploration efficiency. `[SI p.S4]`

Verification (during multi-walker run):

```
# Check all 10 walkers are running:
squeue -u $USER | grep pftrps

# Check HILLS files are being written:
ls -lh $WORK/runs/pftrps_ain_metad/shared_bias/HILLS*
```

```
# Should show 10 files (HILLS.0 through HILLS.9), all growing

# Check COLVAR for each walker:
wc -l $WORK/runs/pftrps_ain_metad/walker_*/COLVAR
# Each walker should have lines accumulating over time
```

Phase 9: FES Reconstruction [DRAFT – not yet verified] (~1 hr)

Goal: Reconstruct the free energy surface (FES) from the accumulated HILLS files and identify the Open, Partially Closed, and Closed basins.

Step 9.1: Merge HILLS files from all walkers

```
cd $WORK/runs/pftrps_ain_metad/shared_bias

# Concatenate all HILLS files (remove duplicate headers)
cat HILLS.0 > HILLS_all
for i in 1 2 3 4 5 6 7 8 9; do
  grep -v "^#" HILLS.${i} >> HILLS_all
done

# Sort by time:
sort -n -k1 HILLS_all > HILLS_all_sorted
```

Step 9.2: Reconstruct FES with sum_hills

What: PLUMED's `sum_hills` utility reads the HILLS file and reconstructs the free energy surface by summing all deposited Gaussian hills with the well-tempered correction.

Key parameter: `--kt 0.695` = k_B * T = 0.001987 kcal/mol/K * 350 K = 0.695 kcal/mol. This is needed for the well-tempered reweighting.

```
module load anaconda/2024.02 && conda activate trpb-md

plumed sum_hills \
  --hills $WORK/runs/pftrps_ain_metad/shared_bias/HILLS_all_sorted \
  --outfile $WORK/analysis/fes.dat \
  --mintozero \
  --kt 0.695
```

sum_hills Flag	Meaning
`--hills`	Input HILLS file
`--outfile`	Output FES grid
`--mintozero`	Shift FES so the global minimum is at 0 kcal/mol
`--kt`	k_B*T for well-tempered correction: 0.695 kcal/mol at 350 K

Step 9.3: Identify basins

State definitions from SI [SI p.S4]:

State	s(R) Range	Description
Open (O)	1–5	COMM domain open, substrate tunnel accessible
Partially Closed (PC)	5–10	Intermediate state
Closed (C)	10–15	COMM domain closed, catalytically competent

Verification:

```
ls -lh $WORK/analysis/fes.dat
# Should exist, ~1-10 MB depending on grid resolution

# Check FES has data:
wc -l $WORK/analysis/fes.dat
# Should have many lines (grid points)

# Find global minimum:
sort -n -k3 $WORK/analysis/fes.dat | head -1
# Shows the (s, z, FES) point with lowest free energy

# Convergence check: remake FES using only first half and second half of HILLS
# If the two FES profiles match within ~1 kcal/mol, the MetaD is converged
```

Step 9.4: Convergence assessment

Method from SI [SI p.S4]: Plot the free energy difference between Open and Closed basins (Delta_G_OC) as a function of accumulated simulation time. When Delta_G_OC plateaus (fluctuations < 1 kcal/mol), the MetaDynamics is converged.

```
# Blockwise FES reconstruction (split HILLS into time blocks):
for block in 10 20 30 40 50; do
  head -n ${block}000 $WORK/runs/pftrps_ain_metad/shared_bias/HILLS_all_sorted \
    > /tmp/hills_block_${block}ns

  plumed sum_hills \
    --hills /tmp/hills_block_${block}ns \
    --outfile $WORK/analysis/fes_block_${block}ns.dat \
    --mintozero \
    --kt 0.695
done
```

Appendix A: Parameter Reference

Complete AMBER .in Parameter Table

Parameter	Value(s) Used	Meaning	Source
`imin`	1	Run energy minimization (not MD)	AMBER manual
`imin`	0	Run molecular dynamics simulation	AMBER manual
`ntx`	1	Read coordinates only from input (no velocities)	AMBER manual
`ntx`	5	Read coordinates AND velocities from restart file	AMBER manual
`irest`	0	Do not restart; begin a new simulation from scratch	AMBER manual
`irest`	1	Restart: continue from previous run using restart file	AMBER manual
`maxcyc`	10000	Maximum number of minimization cycles to attempt	`[operational default]`
`ncyc`	5000	Switch from steepest descent to conjugate gradient after this many cycles. Steepest descent is robust for initial large displacements; conjugate gradient converges faster near the minimum.	`[operational default]`
`nstlim`	50000	Number of MD steps. 50,000 x 0.001 ps = 50 ps (heating steps)	`[SI p.S2]`
`nstlim`	1000000	Number of MD steps. 1,000,000 x 0.002 ps = 2 ns (equilibration)	`[SI p.S3]`

Parameter	Value(s) Used	Meaning	Source
nstlim	250000000	Number of MD steps. 250,000,000 x 0.002 ps = 500 ns (production)	[SI p.S3]
dt	0.001	Timestep in ps (1 fs). Used during heating with restraints for stability.	[operational default]
dt	0.002	Timestep in ps (2 fs). Requires SHAKE constraints on bonds involving H.	[SI p.S3]
ntc	2	SHAKE: constrain all bonds involving hydrogen atoms. Allows 2 fs timestep.	[SI p.S3]
ntf	2	Do not calculate forces for SHAKE-constrained bonds. Must match ntc=2.	[SI p.S3]
ntt	3	Langevin thermostat: adds random forces and friction to maintain temperature. More robust than velocity rescaling for biomolecules.	[SI p.S3]
gamma_ln	1.0	Langevin collision frequency in ps^-1. Controls coupling strength to heat bath. Low values (1.0) give weak coupling; high values (5.0) give strong coupling.	[operational default]
tempi	0.0	Initial temperature for velocity assignment (only in heat1, where we start from 0 K)	AMBER manual

Parameter	Value(s) Used	Meaning	Source
`temp0`	50.0–350.0	Target temperature in K. Varies by heating step; final value 350 K for *P. furiosus* (thermophile).	`[SI p.S2–S3]`
`nmropt`	1	Enable NMR-style restraints. Here used for the &wt section that ramps temperature linearly over the step.	AMBER manual
`ntb`	1	Periodic boundary conditions with constant volume (NVT ensemble). Used for heating and production.	`[SI p.S2–S3]`
`ntb`	2	Periodic boundary conditions with constant pressure (NPT ensemble). Used for equilibration.	`[SI p.S3]`
`ntp`	0	No pressure coupling (NVT).	`[SI p.S2]`
`ntp`	1	Isotropic pressure coupling (NPT). Box dimensions adjust uniformly in all directions.	`[SI p.S3]`
`barostat`	1	Berendsen barostat: weak-coupling pressure control. Good for equilibration; not rigorous for production (does not produce exact NPT ensemble).	`[operational default]`
`pres0`	1.0	Target pressure in bar (~1 atm). Standard ambient pressure.	`[operational default]`

Parameter	Value(s) Used	Meaning	Source
`taup`	1.0	Pressure relaxation time in ps. Controls how quickly the box responds to pressure deviations.	`[operational default]`
`cut`	8.0	Non-bonded interaction cutoff in Angstroms. Interactions beyond this distance are handled by PME (electrostatics) or truncated (van der Waals).	`[SI p.S3]`
`ntr`	1	Apply positional restraints to atoms specified by restraintmask. A harmonic potential pulls restrained atoms toward their reference positions.	`[SI p.S2]`
`ntr`	0	No positional restraints. Atoms move freely.	–
`restraint_wt`	500.0	Restraint force constant for minimization: 500 kcal/mol/A^2. Very strong; atoms barely move from reference.	`[SI p.S2]`
`restraint_wt`	210.0–10.0	Restraint force constant for heating steps. Decreases progressively to allow protein relaxation.	`[SI p.S2]`

Parameter	Value(s) Used	Meaning	Source
restraintmask	'!@H= & !:WAT & !:Na+'	Atom selection for restraints: all atoms EXCEPT hydrogens, water molecules, and Na+ ions. Effectively restrains protein and cofactor heavy atoms.	[SI p.S2]
ntpr	500 / 5000	Print energy information to output file every N steps. Lower values give more monitoring detail.	[operational default]
ntwx	5000	Write trajectory frame every N steps. At dt=0.002, 5000 steps = 10 ps between frames.	[operational default]
ntwe	500 / 5000	Write energy data every N steps.	[operational default]
ntwr	50000 / 500000	Write restart file every N steps. For crash recovery.	[operational default]
ioutfm	1	Use NetCDF binary format for trajectory output. More compact and faster than ASCII.	[operational default]

PLUMED Parameter Table

Parameter	Value	Meaning	Source
LAMBDA	0.033910	PATHMSD smoothing parameter (A^-2). Controls sharpness of frame assignment. Formula: 2.3 / mean_MSD.	Calculated; SI reports 0.029
HEIGHT	0.628	Hill height in kJ/mol. SI: 0.15 kcal/mol x 4.184 = 0.628 kJ/mol.	[SI p.S3]

Parameter	Value	Meaning	Source
PACE	1000	Deposit hill every 1000 steps = every 2 ps (at dt=0.002 ps).	[SI p.S3]
BIASFACTOR	10	Well-tempered bias factor (gamma). Controls effective CV temperature: T_eff = T*(1+1/gamma).	[SI p.S3]
TEMP	350	System temperature in K. Must match GROMACS thermostat.	[SI p.S3]
SIGMA	0.05	Initial Gaussian width for adaptive mode. Overridden by ADAPTIVE.	[UNVERIFIED]
ADAPTIVE	GEOM	Use geometric adaptive Gaussian widths. Adjusts hill shape to local metric tensor of CV space.	[SI p.S3]
WALKERS_N	10	Number of parallel walkers sharing bias.	[SI p.S4]
WALKERS_RSTRIDE	1000	Read shared HILLS every 1000 steps.	[operational default]

GROMACS MDP Parameter Table

Parameter	Value	Meaning	Source
integrator	md	Leap-frog integrator (GROMACS default for MD)	GROMACS manual
dt	0.002	Timestep: 2 fs	[SI p.S3]
nsteps	25000000	25M steps = 50 ns per walker	[SI p.S4]
tcoupl	v-rescale	Velocity-rescale thermostat[5]. Produces correct canonical ensemble.	[operational default]
ref_t	350	Target temperature: 350 K	[SI p.S3]
pcoupl	no	No pressure coupling (NVT)	[SI p.S3]

[5] Bussi, G.; Donadio, D.; Parrinello, M. *J. Chem. Phys.* **2007**, *126*, 014101.

Parameter	Value	Meaning	Source
`coulombtype`	PME	Particle Mesh Ewald for long-range electrostatics	`[SI p.S3]`
`rcoulomb`	0.8	Electrostatic cutoff: 0.8 nm = 8 A	`[SI p.S3]`
`rvdw`	0.8	Van der Waals cutoff: 0.8 nm = 8 A	`[SI p.S3]`
`constraints`	h-bonds	Constrain bonds involving hydrogen (equivalent to SHAKE)	`[SI p.S3]`
`continuation`	yes	Continuing from AMBER equilibration (do not reset time)	–
`gen_vel`	no	Read velocities from input (do not generate)	–

Appendix B: Troubleshooting

Known Failure Patterns

These are errors that have occurred during this project, documented in `replication/validations/failure-patterns.md`:

ID	Problem	Solution
FP-003	PDB residue name wrong (wrote “PLP” instead of “LLP”)	Always extract residue names from actual PDB files, never from memory
FP-004	PLUMED atom indices wrong after GROMACS conversion	Always determine indices from the GROMACS `.gro` file, not from the input PDB
FP-006	`OMP_NUM_THREADS` not set in Slurm script, GROMACS crashes	Add `export OMP_NUM_THREADS=${SLURM_CPUS_PER_TASK}` to all Slurm scripts
FP-007	Wrong conda path in Slurm script	Use `module load anaconda/2024.02 && conda activate trpb-md` on Longleaf
FP-009	AM1-BCC fails for PLP (SCF divergence)	Use RESP charges with Gaussian QM; BCC is prohibited for PLP
FP-010	antechamber misassigns atom types without hydrogens	Run `reduce` to add hydrogens before antechamber
FP-012	Odd electron count -> singlet impossible in Gaussian	Verify `(Z_total - charge) % 2 == 0` before submitting
FP-013	`iop(6/50=1)` causes Gaussian 16 error	Do not use `iop(6/50=1)`; use `-fi gout` in antechamber instead

ID	Problem	Solution
FP-015	`calculate_msd()` returned RMSD, lambda off by 130x	Ensure MSD function returns squared quantity (A^2), not square root
FP-017	antechamber ignores `-rn` flag, uses filename for residue name	Verify mol2 residue name with `grep SUBSTRUCTURE` after antechamber

Common Issues

Q: AMBER pmemd.cuda crashes with "CUDA out of memory" > Reduce system size or request a GPU with more memory. On Longleaf, specify `--gres=gpu:a100:1` for A100 (80 GB) instead of default GPU.

Q: tleap says "Could not find unit AIN" > Make sure you loaded the `.lib` file AND the `.frcmod` file before loading the PDB. The `loadoff` and `loadamberparams` commands must come before `loadpdb`.

Q: Gaussian 16 SCF does not converge > For PLP-like molecules, try: (1) Use `SCF=(MaxCycle=512,XQC)` for extended convergence. (2) Start from a geometry closer to equilibrium. (3) Check the charge and multiplicity are physically reasonable.

Q: GROMACS grompp gives "atom X not found in topology" > The ParmEd conversion may have residue naming issues. Check that all residue names in the `.gro` file match entries in the `.top` file.

Q: PLUMED PATHMSD gives "PDB file does not match" > The reference PDB atom count and naming must exactly match the atoms selected in the running simulation. Regenerate `path.pdb` using the post-conversion GROMACS structure.

Q: MetaDynamics FES does not converge > (1) Run longer (more ns per walker). (2) Check that hills are being deposited (HILLS file should grow). (3) Verify the path CV is capturing the relevant conformational change by plotting s(R) from COLVAR.

Discrepancy Summary

#	Parameter	SI Value	Our Value	Severity	Notes
1	Lambda (PATHMSD)	0.029 A^-2	0.033910 A^-2	HIGH	Different MSD; likely alignment method difference
2	Heat7 restraint	Not specified	10.0 kcal/mol/A^2	MEDIUM	SI lists 6 values for 7 steps
3	SIGMA (PLUMED)	Not reported	0.05	MEDIUM	ADAPTIVE mode self-corrects

#	Parameter	SI Value	Our Value	Severity	Notes
4	Gaussian version	09	16c02	LOW	Same basis set and method
5	AMBER version	16	24p3	LOW	Same ff14SB force field
6	GROMACS version	5.1.2	2026.0	LOW	Same PLUMED interface

Document generated: 2026-04-02 Verified phases: 0–5 (UNC Longleaf) Draft phases: 6–9 (pending execution)

References